
Learning to Remember, Learn, and Forget in Attention-Based Models

Djohan Bonnet¹ Jamie Lohoff^{1,2} Jan Finkbeiner^{1,2} Elidona Shiqerukaj² Emre Neftci^{1,2}

Abstract

In-Context Learning (ICL) in transformers acts as an online associative memory and is believed to underpin their high performance on complex sequence processing tasks. However, in gated linear attention models, this memory has a fixed capacity and is prone to interference, especially for long sequences. We propose Palimpsest, a self-attention model that views ICL as a continual learning problem that must address a stability-plasticity dilemma. Palimpsest uses Bayesian metaplasticity, where the plasticity of each attention state is tied to an importance state grounded by a prior distribution that captures accumulated knowledge. We demonstrate that various gated linear attention models emerge as specific architecture choices and posterior approximations, and that Mamba2 is a special case of Palimpsest where forgetting dominates. This theoretical link enables the transformation of any non-metaplastic model into a metaplastic one, significantly expanding its memory capacity. Our experiments show that Palimpsest consistently outperforms baselines on the Multi-Query Associative Recall (MQAR) benchmark and on Commonsense Reasoning tasks.

1. Introduction

Transformers are the workhorse of generative AI, underpinning breakthroughs in large language modeling (Brown et al., 2020; Chowdhery et al., 2023), computer vision (Dosovitskiy et al., 2021), robotics (Brohan et al., 2022), and scientific domains like chemistry (Schwaller et al., 2019) and biology (Jumper et al., 2021). Central to this success is the *self-attention* mechanism (Vaswani et al., 2017), which allows a model to capture interactions among all tokens in a sequence in a highly parallelizable

manner. However, this mechanism requires caching key-value (KV) pairs for each input token, causing memory and computational costs to grow linearly and quadratically, respectively, with sequence length. Consequently, long-context data scenarios pose strong technical challenges for classical Transformer architectures, especially at the edge (Kim et al., 2023).

A promising direction to simultaneously address the computational and memory bottlenecks is to adopt *fixed-size* attention memories, such as in linear transformers (Schlag et al., 2021) and state space models (Gu & Dao, 2023). These trade the dynamically growing KV cache with a fixed-size memory state that is updated at each token. A key insight is their ability to perform *in-context learning* (ICL) (Brown et al., 2020), whereby the model effectively “learns” at test time by processing contextual examples within the input. ICL essentially tackles a continual and online learning problem, not unlike how the brain dynamically adjusts its weights through neural and synaptic plasticity. In this view, learning with fixed-size memory without revisiting past states and inputs (*i.e.* replay) can lead to catastrophic interference (McCloskey & Cohen, 1989) and further deteriorate with sequence length. This interference contributes to the limited capacity of linear transformers (Schlag et al., 2021).

The concept of *metaplasticity* from neuroscience (Abraham & Bear, 1996) posits that the degree of plasticity is itself adaptive to preserve prior knowledge. This inspired practical algorithms to solve catastrophic forgetting by dynamically adjusting plasticity (Zenke et al., 2017; Kirkpatrick et al., 2017b; Benna & Fusi, 2016). In this work, we pose ICL as a continual learning problem that can be solved through metaplasticity, building on prior work that associated ICL and gradient descent (Akyürek et al., 2023; Von Oswald et al., 2023b). However, existing metaplasticity methods are unsuitable for transformers because they rely either on clearly demarcated tasks and associated labels, growing memory, or lack differentiability (Kudithipudi et al., 2023). As we aim to introduce metaplastic methods embedded in transformer architectures, differentiability is necessary for training. One natural solution to these challenges is to formulate memory updates as a process of *Bayesian Gradient Descent* (BGD), ensuring that each update balances prior knowledge with new evidence (Zeno et al., 2021). BGD adjusts parameters according to their uncertainty in an online

¹Forschungszentrum Jülich, Germany ²RWTH Aachen, Germany. Correspondence to: Djohan Bonnet <d.bonnet@fz-juelich.de>, Emre Neftci <e.neftci@fz-juelich.de>.

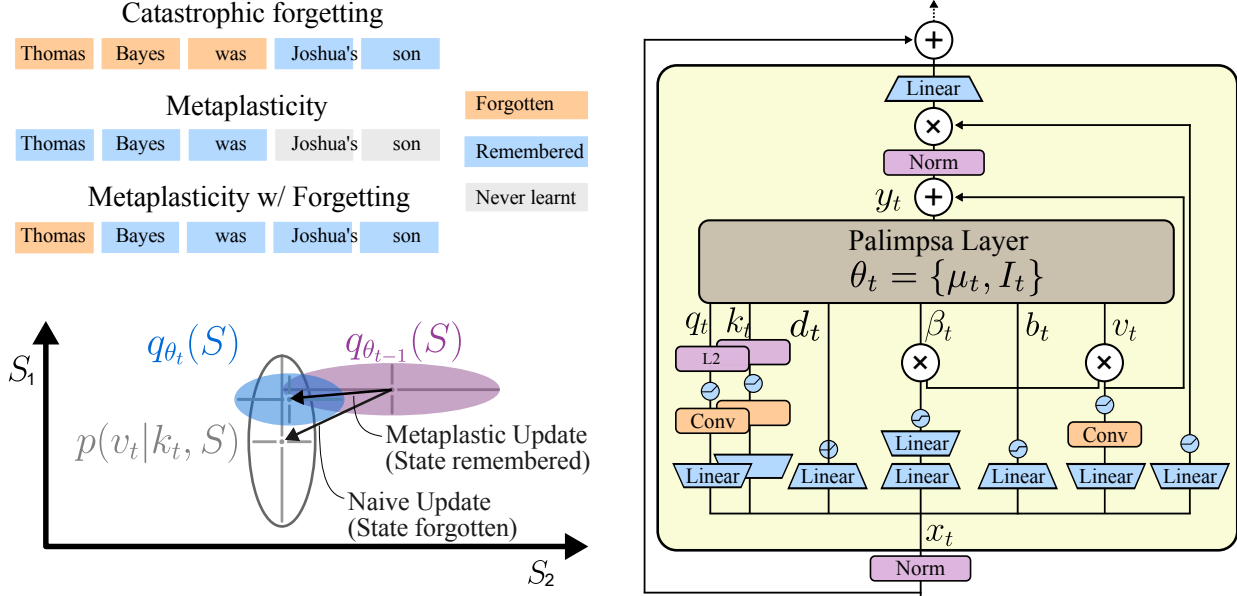


Figure 1. Bayesian Metaplasticity Attention. Self-attention in autoregressive transformers is inherently a continual learning problem, and as such can suffer from catastrophic forgetting. Metaplasticity dynamically modifies the learning rate to preserve important prior information. (Bottom-left) Illustration of Bayesian metalearning: q_{θ_t} is the (variational) distribution over memory states $\mathbf{S}$ at time step t . (Right) We derive Palimpsest, a new attention block based on an online Bayesian posterior, preventing both catastrophic forgetting and remembering using metaplasticity.

fashion. However, BGD and related metaplasticity methods suffer from catastrophic *remembering* (Kaushik et al., 2021), induced by a loss of plasticity that *suppresses* the ability to learn new knowledge to preserve old ones. For ICL, this would imply that the model becomes unable to incorporate new information past a critical sequence length, which is often cited as a key limitation of state space models (Behrouz et al., 2024). The recent Metaplasticity from Synaptic Uncertainty (MESU) solves this problem by leveraging a Bayesian framework to also enable *forgetting* by discarding stale or unused information (Bonnet et al., 2025). The degree of this *palimpsest* property¹ can be adjusted through a Bayesian prior, which effectively controls the time horizon of the memory.

Building on MESU, we introduce *Palimpsest*, a dual-state attention block that performs metaplastic Bayesian updates to a fixed-size attention memory. Palimpsest mitigates in-context catastrophic forgetting by adjusting per-state update magnitude based on memory uncertainty, preserving critical past information. By releasing information predicted as stale, it also prevents in-context catastrophic remembering. The key to deriving Palimpsest is the formulation of self-attention as an optimization of an inner variational free energy at test-time, for which the variational posterior updates can be

¹A palimpsest is a writing surface where the original text is scraped or washed off to be reused, especially used in ancient works when parchment was of limited supply – similarly to the fixed-size memory in Palimpsest

analytically computed. With our custom-developed kernels, Palimpsest scales efficiently on GPUs with speeds comparable to Mamba1 (Gu & Dao, 2023).

Furthermore, we show that existing gated linear models can be derived from specific assumptions on the variational posterior, unifying several prior works within a common mathematical framework. Specifically, we demonstrate a continuum between Mamba2 and Palimpsest, suggesting a hybrid pipeline: large-scale pre-training for efficiency, followed by metaplastic fine-tuning with Palimpsest to expand its memory capacity. We validate our approach on synthetic, commonsense reasoning, and long context benchmarks using two backbones: Palimpsest-D (Deltanet-based) and Palimpsest-M (Mamba2-based). Our results demonstrate that metaplasticity consistently improves performance over non-metaplastic baselines across various memory sizes.

Our specific contributions are:

- The introduction of the concept of metaplasticity in ICL to mitigate in-context catastrophic forgetting.
- Palimpsest, a model that prevents in-context catastrophic remembering by gradually releasing outdated knowledge, with scalable custom kernels.
- A mathematical framework that casts several gated linear attention and state space models as special cases of a common Bayesian framework.
- Methods to transform non-metaplastic models into metaplastic ones.

1.1. Related Work

Continual Learning Methods in Neural Sequence Models Continual Learning (CL) methods can be broadly categorized into complementary mechanisms for CL systems (McClelland et al., 1995) namely *replay-based* ((Buzzega et al., 2020), (Lopez-Paz & Ranzato, 2017)), *dynamic network expansion* ((Rusu et al., 2016), (Wang et al., 2024)), and *regularization-based* methods ((Zenke et al., 2017), (Kirkpatrick et al., 2017a)). CL has been previously considered in State Space Models (SSM) (Cheng et al., 2024; Zhang et al., 2024), but did not study the ICL aspects of this problem. Other work applied modern transformer and SSM models to improve on CL benchmarks (Thengane et al., 2022). To provide a local and efficient method to solve the stability-plasticity dilemma using finite size memory in ICL, we focus specifically on regularization-based CL methods. This allows us to tackle general language modeling tasks without an explicitly constructed CL structure, namely language.

A significant body of related work interprets In-Context Learning (ICL) as Bayesian inference (Xie et al., 2021; Wies et al., 2023; Arora et al., 2024a; Hahn & Goyal, 2023; Zhang et al., 2023; Falck et al., 2024). These studies generally posit that the Transformer architecture implicitly approximates a Bayesian learner in context at the system level, effectively performing inference over latent tasks or concepts. Our approach differs fundamentally, as we explicitly derive a new gated linear attention mechanism directly from Bayesian principles. Bayesian learning and metaplasticity are closely linked in the hypothesis that synapses maintain uncertainty estimates (Aitchison et al., 2021) to dynamically adjust their plasticity. This perspective inspired metaplastic CL methods like MESU (Bonnet et al., 2025). This type of metaplasticity can be written in a local and linear fashion. In Palimpsest, we take advantage of these properties to allow scalable training on GPUs using the techniques employed for gated state-space models (Gu & Dao, 2023).

Gated Linear Recurrence Models and Metaplasticity

This type of metaplasticity is strikingly related to existing gated linear attention models and ICL. (Akyürek et al., 2023; Von Oswald et al., 2023a) demonstrated that the forward pass through the linear attention mechanism can be framed as an iterative gradient descent on a local regression loss function, thus enabling test-time adaptation of the model underpinning ICL. This showed how the choice of the loss function directly influenced the resulting gating and forgetting mechanisms (Yang et al., 2024). The perspective of gradient descent local loss functions was further elaborated in Mesa-optimization and Mesanet (Von Oswald et al., 2023b; 2025), Test-Time Training (Sun et al., 2023), Gated Deltanets (Yang et al., 2025), Longhorn (Liu et al., 2025) and Titans (Behrouz et al., 2024).

Palimpsest shares similarity with Longhorn (Liu et al., 2025) and Mesanet (Von Oswald et al., 2025). Longhorn (Liu et al., 2025) explicitly leverages this online learning relationship to derive its gating mechanisms. While input gating enables the model to control which information is compressed into the memory matrix achieving a form of suboptimal stability-plasticity trade off, it lacks an efficient mechanism to protect important information.

Specifically, Longhorn introduces a trainable, input-dependent importance factor that weights the loss function (which we later refer to as β_t) and dictates the stability-plasticity ratio for each row of the fixed-size memory matrix, effectively freezing a row for a given input when a value is zero. In Longhorn, the plasticity rate remains constant for every memory state within a sequence, meaning there is no in-context metaplasticity.

In contrast, our Bayesian metaplasticity perspective dictates a plasticity rate that changes *in-context*. This is achieved using a second state, as in the Mesanet. Mesanet (Von Oswald et al., 2025) seeks the optimal solution of an in-context regression objective, leading to an update that is strikingly similar to Palimpsest. This is because Bayesian and frequentist solutions to linear regression problems lead to identical estimates of the mean. However, Mesanet addresses a simplified objective where the stability-plasticity ratio is fixed for each memory row. Consequently, every state in a row shares the same learnable plasticity. This prevents true metaplasticity, which requires dynamically adapting the learning rate of every individual state. This structural simplification enables Mesanet to focus on capturing synaptic correlations by computing the inverse covariance matrix using a parallelized, iterative gradient descent. However, this numerical approximation comes at the computational cost of iterative inner-loop updates. To maintain throughput and the spirit of metaplasticity, where synaptic learning rates are independent, Palimpsest relies on a diagonal approximation of the covariance matrix. By using the resulting variance as a learning rate, Palimpsest closely aligns with the neuroscience model proposed by (Aitchison et al., 2021).

Finally, because Palimpsest is inherently Bayesian, the attention head provides output uncertainty that could be leveraged in future work to improve overall model performance.

2. Background and Methods

2.1. Background: Self-Attention Heads and Metaplastic Memory

The decoder self-attention (Vaswani et al., 2017) autoregressively maps the sequence $\{x_t\}_{t=1}^L$ into $\{y_t\}_{t=1}^L$. Each token x_t is projected into key, query, and value vectors $k_t, q_t \in \mathbb{R}^{d_k}, v_t \in \mathbb{R}^{d_v}$. Self-attention computes a weighted combination of the values $V_t = [V_{t-1}, v_t]$ based on the similarity

between each query q_t and the keys $K_t = [K_{t-1}, k_t]$. Concretely, this can be expressed as:

$$y_t = V_t \text{Softmax}(K_t^\top q_t).$$

The similarity between queries and keys (the ‘‘look-up’’ step) determines the extent to which each value contributes to the output. Linear Transformers (Schlag et al., 2021) omit the softmax, in which case the KV cache can be replaced by a fixed-size attention matrix, updated in-place for every new incoming token with a ‘‘Hebbian’’ update:

$$S_t = S_{t-1} + v_t \otimes k_t, \text{ and } y_t = S_t q_t, \text{ where } S_t \in \mathbb{R}^{d_v \times d_k}.$$

However, if the new key k_t is not orthogonal to previously stored keys, the update will partially overwrite older memories. Schlag et al. demonstrated the limited capacity of linear transformers and implemented an error-correcting delta rule to mitigate this issue. This idea has been later expanded to improve performance and trainability (Sun et al., 2024; Liu et al., 2025; Yang et al., 2025). Given this interference, should the model prioritize learning to map new key–value pairs, or preserve the relationships from earlier pairs?

Recently, Bonnet et al. proposed MESU, a mathematically grounded approach to solve such dilemma from a Bayesian framework. MESU formalizes forgetting to prevent a vanishing learning rate thus avoiding catastrophic remembering (Kaushik et al., 2021). Building on this idea, we treat the self-attention memory update as a Bayesian inference problem that includes formalized forgetting.

2.2. Derivation of Palimpse

Bayesian Formulation of the Attention Objective For a given time step, the attention head can be defined as $p(v|k, \beta, S)$. Given the inputs $k \in \mathbb{R}^{d_k}$ and importance factor $\beta \in \mathbb{R}^{d_v}$, the attention head models the distribution of values $v \in \mathbb{R}^{d_v}$ conditioned on the state $S \in \mathbb{R}^{d_v \times d_k}$. Using Gaussian assumptions, the objective function (linear regression) of the attention head is defined as the negative log-likelihood:

$$-\log p(v | k, \beta, S) = \frac{1}{2} \|S k - v\|_{\text{diag}(\beta)}^2.$$

Given t tokens $\{x_j\}_{j=1}^t$ with $x_j \in \mathbb{R}^{d \times 1}$, we obtain t data points $\{d_j\}_{j=1}^t = \{(k_j, \beta_j), v_j\}_{j=1}^t$ to minimize the above loss ‘‘in-context’’. Applying Bayes’ rule gives the posterior distribution on S , which can be expressed in the recursive form:

$$p(S|d_{1:t}) = \frac{p(d_t|S) \cdot p(S|d_{1:t-1})}{p(d_t|d_{1:t-1})}. \quad (1)$$

Since q_t is independent of S , the output y_t can be defined as:

$$y_t = \mathbb{E}_{p(S|d_{1:t})} [S q_t] = \mathbb{E}_{p(S|d_{1:t})} [S] q_t = \mu_t q_t. \quad (2)$$

Catastrophic remembering occurs when the prior $p(S|d_{1:t-1})$ becomes overly concentrated as t increases, causing the likelihood $p(d_t|S)$ to have a negligible impact on the posterior, inhibiting the integration of new information. This can be prevented by truncating the posterior to the last N tokens:

$$p(S|d_{t-N+1:t}) \propto \underbrace{p(d_t|S)p(S|d_{t-N:t-1})}_{\text{Learning}} \cdot \underbrace{\frac{1}{p(d_{t-N}|S)}}_{\text{Forgetting}}$$

where N now acts as a memory window size (Bonnet et al., 2025). In an online scenario, $p(d_{t-N}|S)$ is unavailable at time t . Thus, we approximate this truncation by the weighted posterior (See Appendix A.2):

$$p_w(S|d_{1:t}) \propto p(d_t|S) p_w(S|d_{1:t-1}) \cdot \left(\frac{p_w(S|d_{1:t-1})}{p(S)} \right)^{-\frac{1}{N}}$$

We refer to p_w as the weighted posterior because the influence of historical data decays over time. This suggests a simple interpretation: at each time step, we discard a fraction $\frac{1}{N}$ of the accumulated prior weight. Effectively, the weighted posterior never carries more weight than that of a truncated posterior with a memory window of size N . Thus, with a context window N that reflects the maximum memory capacity of the attention head, catastrophic remembering is avoided. Inspired by common practice in state space models, we introduce input-dependence to this memory window, denoting it N_t . Our Bayesian formulation naturally defines the forgetting gate as $\alpha_t := (1 - \frac{1}{N_t})$.

To align with the parameterization of current state-of-the-art architectures (Gu & Dao, 2023), we express this gate as $\alpha_t = \exp(-A d_t)$, where $d_t \in \mathbb{R}_+$ represents the input-dependent step size and $A \in \mathbb{R}_+$ is a learnable parameter, ensuring $\alpha_t \in (0, 1]$. This explicit mapping provides a Bayesian interpretation of the effective dependency range of the attention head.

Bayesian Attention as an Optimization Problem While the weighted posterior $p_w(S|d_{1:t})$ is analytically tractable in our setting, we deliberately employ Variational Inference (VI) to cast the update as an optimization problem (Blundell et al., 2015). Crucially, in this conjugate setting, VI recovers the exact solution. We adopt this formulation to demonstrate that various gated linear attention models emerge as specific choices of architecture and posterior approximations. For each time step t , we define $q_{\theta_{t,i}}(S_i)$, $i = 1, \dots, d_v$, a variational distribution over S_i parameterized by $\theta_{t,i}$. We minimize the Kullback–Leibler (KL) divergence between q_{θ_t} and the true posterior by optimizing $\theta_{t,i}$:

$$\theta_{t,i} = \underset{\theta}{\operatorname{argmin}} D_{\text{KL}} [q_{\theta}(S_i) \| p_w(S_i|d_{1:t})], \quad i = 1, \dots, d_v.$$

Here, $q_{\theta_{t,i}}$ is modeled as a multivariate Gaussian of dimension d_k :

$$q_{\theta_{t,i}}(S) \sim \mathcal{N}(\mu_{t,i}, \Sigma_{t,i}), \text{ where } \theta_{t,i} = \{\mu_{t,i}, \Sigma_{t,i}\}, \\ \mu_{t,i} \in \mathbb{R}^{d_k}, \quad \Sigma_{t,i} \in \mathbb{R}^{d_k \times d_k}, \quad i = 1, \dots, d_v.$$

Using Bayes' theorem and the definition of KL divergence, finding the optimal $\theta_{t,i}$ is equivalent to minimizing the free energy:

$$\mathcal{F}_{t,i} = D_{\text{KL}}[q_{\theta_{t,i}}(S_i) \| p(S_i)] - \mathbb{E}_{q_{\theta_{t,i}}(S_i)} \left[\sum_{s=1}^t \log p(d_s | S_i) \right].$$

With Gaussian assumptions, $\mathcal{F}_{t,i}$ decomposes into three components, and a term C_{Σ} that depends solely on the covariance:

$$\mathcal{F}_{t,i}(\mu_i) = \underbrace{\frac{\beta_{t,i}}{2} \|\mu_i^T \mathbf{k}_t - v_{t,i}\|^2}_{\text{plasticity}} + \underbrace{(1 - \alpha_t) \frac{\|\mu_i\|^2}{2\sigma_{\text{prior}}^2}}_{\text{forgetting}} \\ + \underbrace{\frac{1}{2} \alpha_t (\mu_{t-1,i} - \mu_i)^T \Sigma_{t-1,i}^{-1} (\mu_{t-1,i} - \mu_i)}_{\text{stability}} + C_{\Sigma}. \quad (3)$$

where $\alpha_t := (1 - \frac{1}{N_t})$. The first term contributes to adding new knowledge and is similar to other fixed-term memories. The second introduces the learning window. Through its dependence on N , it determines how far in the past the memories are stored, and then forgotten. The third determines the plasticity rate based on the importance of the synapse. $\mathcal{F}_{t,i}$ is analytically tractable, and its minimum with respect to $\mu_{t,i}$ and $\Sigma_{t,i}$ can be computed in closed form by setting its gradient to zero without requiring any approximation (see Appendix A.3).

Palimpse Layer and Architecture To derive Palimpse we use the reparameterization trick: $S_i = \mu_i + A_i \epsilon_i$, with $A_i A_i^T = \Sigma_{t,i}$, where $\epsilon_i \sim \mathcal{N}(0, \mathbb{I})$. We find the optimal parameters $\mu_{t,i}$ and $\Sigma_{t,i}$, by setting the corresponding gradients to zero:

$$\frac{\partial \mathcal{F}_{t,i}}{\partial \mu_{t,i}} = \vec{0}, \quad \text{and} \quad \frac{\partial \mathcal{F}_{t,i}}{\partial A_{t,i}} = \mathbf{0}. \quad (4)$$

For computational tractability, Palimpse only keeps the diagonal term of the covariance matrices ($\Sigma_{t,i} = \text{diag}(\sigma_{t,i}^2)$). Under these conditions, the solution to Eqs. (4) results in the following **Palimpse update equation**:

$$\mathbf{I}_t = \alpha_t \mathbf{I}_{t-1} + (1 - \alpha_t) \mathbf{I}_{\text{prior}} + \beta_t \otimes \mathbf{k}_t^2 \\ \mu_t = \alpha_t \frac{\mathbf{I}_{t-1}}{\mathbf{I}_t} \odot \mu_{t-1} + \frac{1}{\mathbf{I}_t} \odot [(\beta_t \odot \mathbf{v}_t) \otimes \mathbf{k}_t], \quad (5)$$

where $\mathbf{I}_t := \frac{1}{\sigma_t^2}$ a precision matrix representing the importance of each state, and $\mathbf{I}_{\text{prior}}$ is the importance prior. The full derivation can be found in Appendices A.3–A.5. This update rule is scalably computed chunk-wise using an associative scan (see Appendix A.12). Our experiments utilize

two Palimpse variants. The first, Palimpse-D, serves as a drop-in replacement for the Gated Delta Rule (Yang et al., 2025), differing only by an additional projection for β_t and an intra-layer residual term. The second, Palimpse-M, builds on Mamba2 (Dao & Gu, 2024), where the difference lies in the definition of β_t . Consequently, the metaplasticity ablation Palimpse-M (w/o Meta) is equivalent to Mamba2 (see also next section). A crucial distinction in Palimpse-M is that the term d_t , used to define the forgetting gate $\alpha_t = e^{-Ad_t}$, also functions as an input gate; this implies that new information cannot be integrated without forgetting. Further architectural details are provided in Appendix A.1.

2.3. Bayesian Inference at Test-Time as a General Framework of Gated Models

Here, we revisit prior gated models in the light of the Bayesian view of ICL.

A major challenge in minimizing $\mathcal{F}_{t,i}$ is calculating the inverse of the matrix $\Sigma_{t-1,i}^{-1} \in \mathbb{R}^{d_k \times d_k}$. Von Oswald et al. use the Sherman-Morrison formula to iteratively compute it, but this approach doesn't scale well. Another approach used in Mesanet is to use conjugate gradients (Von Oswald et al., 2025). However, both methods require $\beta_t \in \mathbb{R}$ (a scalar) constant for all i for tractability, since a vector β_t would require the inversion for every row of the state matrix. While using a scalar β_t simplifies inversion, it forces the stability term in Eq. 3 to be the same for every i , preventing per-parameter plasticity.

Longhorn, Deltanets and Palimpse use at least one approximation for tractability. A common simplification is to assume the matrix $\Sigma_{t-1,i}^{-1}$ is diagonal (meaning it only has values on its main diagonal). Both Longhorn and Palimpse leverage this diagonal simplification to support a vector-valued β_t . In contrast, Deltanets solve their objective by assuming the loss is linear around the current states. In Palimpse, we designate this diagonal $\mathbf{I}_{t-1,i} \in \mathbb{R}^{d_k}$ as the ‘‘importance’’ of a given synapse because its inverse provides its optimal learning rate. Tab. 1 summarizes the resulting update rules derived from these specific architecture choices and posterior approximations. For all gated models but Palimpse and Mesanet in Tab. 1, the importance is fixed, so $\mathbf{I}_{t-1,i} = \vec{1}$. In other words, their stability term is constant, meaning they have no metaplasticity.

This analysis also reveals that Mamba2 is a special case of Palimpse. In the Palimpse update (Eq. 5), when the forgetting rate is high, the posterior uncertainty remains close to the initial prior, and $\mathbf{I}_t \cong \mathbf{I}_{\text{prior}}$ is constant. The update rule then simplifies to:

$$\mu_t = \alpha_t \mu_{t-1} + \frac{\beta \odot \mathbf{v}_t}{\mathbf{I}_{\text{prior}}} \otimes \mathbf{k}_t,$$

which takes the same form as Mamba2’s update rule. While Mamba2 was described as solving a negative inner-product loss (Yang et al., 2025), our Bayesian framework reveals Mamba2 as an asymptotic special case of Palimpsest, where forgetting is so strong that the importance matrix is negligible.

2.4. Fine-tuning Mamba2 with Metaplasticity

We exploit the above-described continuum between Mamba2 and Palimpsest to introduce the ability to upgrade any pre-trained Mamba2 model into a metaplastic Palimpsest model via fine-tuning. For 760M models Mamba2 is $> 3\times$ faster than Palimpsest. While this slowness hinders the training of large Palimpsest models from scratch, the fine-tuning ability mitigates this cost and makes full training unnecessary.

Analytically, Palimpsest matches Mamba2 when $\mathbf{I}_t \cong \mathbf{I}_{prior}$. We achieve this condition using the reparameterization $\mathbf{v}_t^* = \beta_t \odot \mathbf{v}_t = \text{SiLU}(\theta_v \mathbf{x}_t)$. By defining the Palimpsest Kernel input as $\mathbf{v}^*$, the β_t multiplication in the kernel becomes unnecessary. This allows us to fix β_t arbitrarily close to zero, ensuring $\mathbf{I}_t \cong \mathbf{I}_{prior}$, without forcing $\mu_t \approx 0$. To do so we introduce a per-head learnable (but fixed in-context) b_{scale} that scales the amplitudes of β_t . For standard training, $b_{scale} = 1$; for fine-tuning, b_{scale} is initialized in $[0.1, 1.0]$ so that β_t starts small placing the model closer to the Mamba2 limit. Intuitively, when tokens are of arbitrarily low importance but carry arbitrarily large values, Palimpsest becomes Mamba2. Thanks to this reparameterization, training can continuously deviate from Mamba2 model towards Palimpsest, as it starts learning that β_t should grow to exploit metaplasticity.

3. Experiments

3.1. Synthetic Experiments: MQAR

To evaluate Palimpsest’s ability to manage and recall states in challenging memorization tasks prone to overwriting, we conduct experiments using the synthetic Multi Query Associative Recall (MQAR) benchmark (Arora et al., 2024b). In this task, the model is presented with pairs of key-value symbols and must retrieve the correct value associated with a specific key at test time. To evaluate the memory capacity of the model rather than its ability to select for important tokens, we selected a configuration that utilizes the maximum number of KV pairs ($L/4$ which is the upper limit imposed by MQAR with sequence length L). Similarly to (Beck et al., 2024), training follows a curriculum: it begins on sequences of length 128 and every 20 epochs progresses to lengths of 256, 512, and 1024. Following prior MQAR benchmarks, the trained models consisted of two gated linear attention layers with a hidden dimension of 128 and a

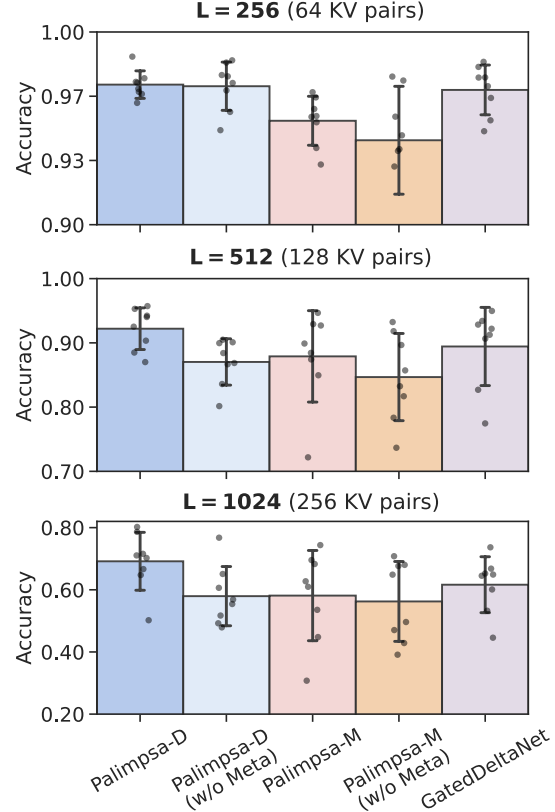


Figure 2. **Curriculum MQAR experiments.** Accuracy averaged over 8 seeds for the best learning rate per model. Individual run accuracies are shown as black dots; error bars represent ± 1 standard deviation. Task difficulty increases with sequence length L . “w/o Meta” indicates that metaplasticity was disabled for those models.

value dimension expansion factor of 2. To make the task more challenging, we reduce the available state size by utilizing 8 heads. We isolate the effect of metaplasticity by evaluating Palimpsest-D and Palimpsest-M and their ablations without their metaplasticity (noting that Palimpsest-M without metaplasticity is equivalent to Mamba2, as previously explained). We include Gated Deltanet as a baseline for comparison. Results are reported over 8 runs per model using different seeds for both initialization and data generation. We explored several learning rates distributed in log space from 10^{-3} to 10^{-2} , and selected the optimal learning rate for each model across seeds and sequence sizes. Results shown in Fig. 2 indicate that metaplasticity consistently improves average performance for both Palimpsest-D and Palimpsest-M. The performance gap between metaplastic and non-metaplastic variants increases with sequence length, particularly for Palimpsest-D. This suggests that increasing task complexity induces the model to leverage metaplasticity to mitigate overwriting. Overall, Palimpsest-D is the best performing model, followed by Gated Deltanet.

Table 1. Comparison of gated attention models from the metaplasticity perspective

Layer	Stability Matrix	Input Gating	Forgetting	Objective resolution
Longhorn	$\mathbb{I}_{d_k}$	$\beta_t \in \mathbb{R}^{d_v}$	None	Diagonal approximation
	Update Rule: $\mu_t = (\mathbb{I}_{d_v \times d_k} - \frac{\beta_t \otimes k_t^2}{1 + \beta_t k_t^\top k_t}) \odot \mu_{t-1} + \frac{\beta_t \odot v_t}{1 + \beta_t k_t^\top k_t} \otimes k_t$			
Deltanet	$\mathbb{I}_{d_k}$	$\beta_t \in \mathbb{R}$	None	First order approximation
	Update Rule: $\mu_t = \mu_{t-1}(\mathbb{I}_{d_k} - \beta_t k_t k_t^\top) + \beta_t v_t k_t^\top$			
Gated Deltanet	$\mathbb{I}_{d_k}$	$\beta_t \in \mathbb{R}$	$\alpha_t \in \mathbb{R}$	First order approximation
	Update Rule: $\mu_t = \mu_{t-1}(\alpha_t(\mathbb{I}_{d_k} - \beta_t k_t k_t^\top)) + \beta_t v_t k_t^\top$			
Mesa	Σ_t^{-1}	$\beta_t \in \mathbb{R}$	$\alpha_t \in \mathbb{R}$	Conjugate grad. approx.
	Update Rule: $\Sigma_t^{-1} = \alpha_t \Sigma_{t-1}^{-1} + (1 - \alpha_t) I_{prior} + \beta_t k_t k_t^\top$ $\mu_t = [\alpha_t \mu_{t-1} \Sigma_{t-1}^{-1} + \beta_t v_t k_t^\top] \Sigma_t$			
Palimpsa	$\text{diag}(I_{t-1,i})$	$\beta_t \in \mathbb{R}^{d_v}$	$\alpha_t \in \mathbb{R}$	Diagonal approximation
	Update Rule: $I_t = \alpha_t I_{t-1} + (1 - \alpha_t) I_{prior} + \beta_t \otimes k_t^2$, $\mu_t = \alpha_t \frac{I_{t-1}}{I_t} \odot \mu_{t-1} + \frac{1}{I_t} \odot [(\beta_t \odot v_t) \otimes k_t]$			
Palimpsa $ \frac{I_t - I_{prior}}{I_{prior}} \ll 1$	Update Rule: $\mu_t = \alpha_t \mu_{t-1} + \frac{\beta_t \odot v_t}{I_{prior}} \otimes k_t$ ($\cong$ Mamba2, see text)			

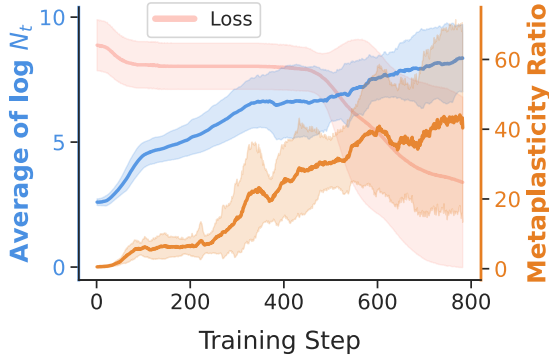


Figure 3. **Palimpsa’s Learning Dynamics:** Memory window N_t (blue), averaged over the context length, and the metaplasticity ratio (orange), defined on the final state importance as $(I_{\max} - I_{\min})/I_{\min}$, and the training loss (pink). A higher ratio indicates stronger differentiation between plastic and consolidated synapses. Shaded regions represent the standard deviation over 8 seeds.

3.2. Palimpsa’s Learning Dynamics on MQAR

As derived in the methods section, the forgetting gate implies an effective memory window N_t via the relation $\alpha_t = (1 - \frac{1}{N_t}) = e^{-Ad_t}$. In Fig. 3, we track the evolution of N_t and the *metaplasticity ratio* during MQAR training, defined as $(I_{\max} - I_{\min})/I_{\min}$ (noting that $I_{\min} > I_{prior} > 0$

holds). The exponential growth of N_t in early epochs demonstrates that the meta-learner correctly identifies that solving MQAR requires slow forgetting (large N_t). When N_t becomes large enough for the task, the training loss starts to decrease. This metaplasticity ratio rises to ≈ 40 , meaning highly consolidated synapses become 40-fold more stable than plastic ones. Notably, during the first 200 steps, the ratio remains low because N_t is small: high forgetting prevents the accumulation of importance, keeping the model in a regime equivalent to Mamba2.

3.3. Language Tasks: Fineweb-Edu and Commonsense Reasoning

We pre-train Palimpsa variants, Gated Deltanets, and a Transformer on the Fineweb-edu dataset at academic scales for two different sizes, 170M and 760M parameters, and for 15B and 30B tokens, respectively. We did not include Mesanet in our benchmarks as, at the time of writing, their public code intended for GPU led to numerical instabilities and their reported results based on SlimPajama. We evaluate each pre-trained model on language modeling and Commonsense Reasoning. Following prior work (Von Oswald et al., 2023b; Liu et al., 2025; Yang et al., 2025), we test Wikitext perplexity and zero shot performance on a range of tasks such as LAMBADA (Paperno et al., 2016),

Table 2. Performance of Palimpsest variants and baselines on language modeling and Commonsense Reasoning tasks. The best results are in bold. The best results within a subgroup are underlined. ppl: perplexity, acc: accuracy, acc_n: normalized accuracy.

Model	Wiki. ppl ↓	LMB. ppl ↓	LMB. acc ↑	PIQA acc_n ↑	Hella. acc_n ↑	Wino. acc ↑	ARC-e acc ↑	ARC-c acc_n ↑	SIQA acc ↑	Avg. acc ↑
<i>170M Model Variants</i>										
Transformer++	29.51	44.90	32.12	65.18	37.64	52.17	55.56	26.19	37.77	43.80
Gated Deltanet	29.16	41.87	30.91	64.91	<u>38.18</u>	48.86	<u>56.73</u>	<u>26.37</u>	39.20	43.59
Palimpsest-D (w/o Meta)	30.80	46.83	27.89	65.02	37.97	52.72	57.95	26.79	39.00	43.91
Palimpsest-D (Fine-tuned 1BT)	<u>29.73</u>	40.26	29.26	65.45	38.41	51.62	58.04	26.02	38.89	43.96
Palimpsest-D (Fully Trained)	30.81	42.27	<u>29.46</u>	65.13	38.02	50.91	55.85	25.77	<u>39.00</u>	43.45
Palimpsest-M (w/o Meta)	31.64	48.42	28.08	<u>65.40</u>	37.25	50.04	55.47	26.02	38.02	42.90
Palimpsest-M (Fine-tuned 1BT)	<u>30.58</u>	<u>42.01</u>	29.83	65.02	37.81	<u>51.38</u>	56.10	25.68	<u>38.54</u>	43.48
Palimpsest-M (Fully Trained)	31.40	43.02	<u>30.62</u>	64.91	<u>38.00</u>	50.28	<u>56.48</u>	<u>26.71</u>	38.02	<u>43.57</u>
<i>760M Model Variants</i>										
Transformer++	18.91	18.84	<u>42.03</u>	70.40	51.14	54.93	66.58	33.70	40.17	51.28
Gated Deltanet	<u>19.06</u>	<u>17.06</u>	41.74	71.33	51.74	<u>55.41</u>	<u>67.30</u>	32.17	40.84	<u>51.50</u>
Palimpsest-D (w/o Meta)	19.33	16.54	41.98	70.40	51.46	55.64	<u>67.93</u>	34.47	40.89	51.82
Palimpsest-D (Fine-tuned 2BT)	<u>19.02</u>	14.86	43.55	<u>71.06</u>	<u>51.63</u>	<u>56.12</u>	<u>67.68</u>	34.64	41.20	52.27
Palimpsest-M (w/o Meta)	19.96	<u>16.34</u>	42.15	70.73	50.61	57.06	67.47	32.76	39.92	51.53
Palimpsest-M (Fine-tuned 2BT)	<u>19.60</u>	16.35	<u>42.17</u>	<u>71.06</u>	<u>50.91</u>	<u>56.99</u>	67.97	<u>34.39</u>	<u>41.61</u>	<u>52.16</u>
<i>2.7B Model Variants</i>										
Palimpsest-M (w/o Meta)	14.46	4.79	62.82	75.73	68.12	63.85	60.06	39.16	44.78	59.22
Palimpsest-M (Fine-tuned 15BT)	14.51	4.79	63.26	76.17	68.58	65.19	63.43	39.33	44.47	60.06

PIQA (Bisk et al., 2020), HellaSwag (Zellers et al., 2019), WinoGrande (Sakaguchi et al., 2021), ARC (Clark et al., 2018), and SIQA (Sap et al., 2019).

We used the Flame framework to train models from scratch (Zhang & Yang, 2025). We also fine-tuned the non-metaplastic Palimpsest variants into metaplastic ones, starting from 14BT, thereby ensuring the same token budget. At this scale, Palimpsest-D (fine-tuned) emerges as the overall winner. As a limitation, fully trained Palimpsest-D underperforms its fine-tuned counterpart. Learning metaplasticity from a random initialization is harder than starting from a checkpoint that already uses basic plasticity.

While fine-tuning benefits both architectures, the performance gain is more pronounced for Palimpsest-M. This was expected as Palimpsest-M does not have a gated MLP, and therefore incorporates more Palimpsest-M layers for the same parameter budget across all models we evaluated. Interestingly, metaplastic models demonstrate superior performance on LAMBADA perplexity. Since LAMBADA requires predicting the final word based on broad context, this improvement is consistent with the better memory retention expected from the metaplastic model.

For the 760M parameter scale, all models were trained for a total of 30B tokens. Metaplastic variants were fine-tuned for 2B tokens starting from the 28B token checkpoint of their non metaplastic counterparts to maintain training budget

parity. Since the fine-tuned model emerged as the winner for the smaller model, we concentrated our computing resources on the fine-tuning of larger metaplastic models.

The results at 760M confirm the impact of metaplastic state at medium scale: Palimpsest-D and Palimpsest-M take the top two spots. The fine-tuned metaplastic models improve upon their respective baselines by approximately 0.6 points in average accuracy, with Palimpsest-D outperforming the Gated Deltanet baseline by nearly 0.8 points (52.27 vs 51.50). Beyond average accuracy, Palimpsest expands memory capacity significantly. This is supported by the per-token NLL analysis on the 760M models (Fig. 4), where Palimpsest variants degrade more slowly than their non-metaplastic counterparts at large token positions, processing nearly $5\times$ more tokens before underperforming the Transformer.

3.4. Large Scale and Long Context Experiments

To evaluate Palimpsest at large scale and address long-context sequence processing, we perform a model surgery on a 2.7B parameter Mamba2 model pre-trained on 300B tokens. Because Mamba2 acts as a limiting case of Palimpsest, we can calculate the effective memory window N_t for each layer. As shown in Fig. 3, the metaplasticity ratio rises with the memory window, suggesting Palimpsest should have the strongest impact on layers with the weakest forgetting. Therefore, we replace the 8 Mamba2 layers exhibiting the

Table 3. LongBench overall results alongside RULER Variable Tracing (VT) performance at extended context lengths for the 2.7B parameter models. Best results are in bold.

Model	LongBench						RULER (Variable Tracing)			
	easy	hard	short	medium	long	overall	4k	8k	16k	32k
Palimpsest-M (w/o Meta)	22.4	17.4	18.9	19.5	19.4	19.3	59.6	49.7	26.0	14.9
Palimpsest-M (Fine-tuned 15BT)	22.4	18.6	16.1	23.3	20.4	20.4	33.5	24.9	22.5	24.4

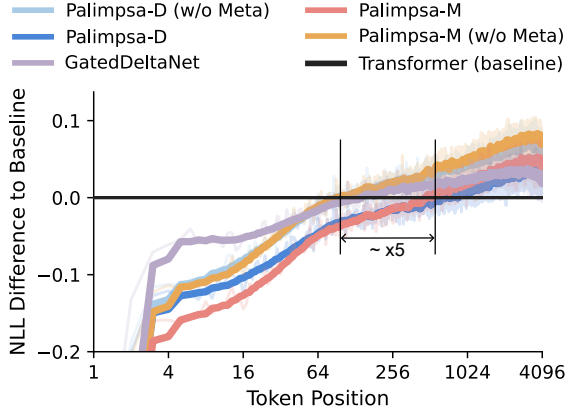


Figure 4. **Per-token NLL difference to a Transformer baseline on WikiText for the 760M parameter models:** Negative values indicate better performance than the Transformer at that token position. All models initially outperform the Transformer at short contexts, but degrade at longer ones. Palimpsest models (with metaplasticity) degrade more slowly, showing that metaplasticity significantly expands memory capacity.

largest N_t with Palimpsest layers. We train this hybrid model using a multi-stage fine-tuning pipeline: continued pre-training for 10B tokens, context extension up to 32k tokens for 5B tokens (Gao et al., 2025; Agarwalla et al., 2024), and a final instruction fine-tuning stage (Chen et al., 2024). To guarantee a fair comparison, we concurrently applied the exact same training recipe to the Mamba2 baseline. Further details are provided in Appendix A.10.

As detailed in Table 2, this upgraded 2.7B Palimpsest-M variant outperforms the Mamba2 baseline by +0.84 points in average commonsense reasoning accuracy. This surpasses the gains observed at smaller scales (+0.58 points at 170M, and +0.63 points at 760M). For long-range dependencies (see Table 3 and Appendix A.11), Palimpsest achieves a +1.1 point overall improvement on the LongBench benchmark (Bai et al., 2025). This advantage is most pronounced in the Medium category (32k–128k context lengths), where Palimpsest dominates by nearly 4 points. Conversely, on RULER Variable Tracing (Hsieh et al., 2024) at shorter contexts (4k–16k), the metaplastic variant underperforms the Mamba2 baseline. However, Palimpsest exhibits lower degradation as sequence length increases, surpassing the baseline at 32k context lengths and confirming its superior capacity to retain information in the long-context regime.

4. Conclusion

We introduced Palimpsest, a Bayesian attention mechanism that reframes In-Context Learning as a continual learning problem within fixed-size memories derived from a variational free energy objective. Palimpsest implements metaplasticity: the plasticity of each of its memory states is dynamically adjusted according to a tracked importance. This approach resolves the stability-plasticity dilemma by protecting critical prior knowledge while allowing the model to forget stale information, effectively preventing both catastrophic forgetting and remembering *in-context*.

Theoretically, our framework unifies disjoint architectures, revealing that Mamba2 is a special case of Palimpsest where forgetting dominates. This insight yields a practical, scalable training pipeline: we exploit the computational speed of non-metaplastic kernels for large-scale pre-training, followed by a metaplastic fine-tuning phase, thereby enabling the use of metaplastic gated linear attention models at any scale.

Empirically, Palimpsest enables better management of state memory in constrained settings, which we show using systematic ablations of the metaplasticity. On the synthetic MQAR benchmark, Palimpsest consistently outperforms baselines, with performance gaps widening as sequence length increases. These gains translate to large-scale language modeling settings: on Commonsense Reasoning benchmarks at the 760M parameter scale, Palimpsest-D achieves an average accuracy of 52.27%, surpassing the Gated Deltanet baseline by nearly 0.8 points. Gains are particularly notable on tasks requiring broad context, such as LAMBADA, validating the model’s superior capacity to retain long-range dependencies. This trend is further supported by our per-token NLL analysis (Figure 4), where Palimpsest variants degrade more slowly than their non-metaplastic counterparts as token positions grow. We demonstrate this recipe at the 2.7B parameter scale, where a surgical replacement of the 8 weakest-forgetting Mamba2 layers with Palimpsest blocks improves average commonsense reasoning accuracy by +0.84 points and overall LongBench performance by +1.1 points. This directly validates metaplasticity as a mechanism for extending the effective memory of pre-trained linear attention models without retraining from scratch.

Impact Statement

Gated linear recurrent models offer a memory-efficient alternative to standard Transformers, making them inherently suitable for edge deployment due to their fixed-size inference state. However, this finite capacity often leads to information loss in long contexts. Palimpsa addresses this limitation by introducing Bayesian metaplasticity, which optimizes how information is learned, remembered, and forgotten within these strict memory bounds.

By enhancing the capabilities of fixed-size memory models, this work supports the deployment of performant sequence modeling on resource-constrained systems. Furthermore, Palimpsa’s local update rules align well with emerging efficient hardware architectures, effectively contributing to the development of low-latency, energy-efficient intelligence at the edge.

Code Availability

Our code is available at <https://github.com/djo1996/Palimpsa>.

Acknowledgements

This work was supported by the Horizon Europe program (EIC Pathfinder METASPIN, grant number 101098651).

References

- Abraham, W. C. and Bear, M. F. Metaplasticity: the plasticity of synaptic plasticity. *Trends in neurosciences*, 19(4): 126130, 1996.
- Agarwalla, A., Gupta, A., Marques, A., Pandit, S., Goin, M., Kurtic, E., Leong, K., Nguyen, T., Salem, M., Alistarh, D., et al. Enabling high-sparsity foundational llama models with efficient pretraining and deployment. *arXiv preprint arXiv:2405.03594*, 2024.
- Aitchison, L., Jegminat, J., Menendez, J. A., Pfister, J.-P., Pouget, A., and Latham, P. E. Synaptic plasticity as bayesian inference. *Nature neuroscience*, 24(4):565–571, 2021.
- Akyürek, E., Schuurmans, D., Andreas, J., Ma, T., and Zhou, D. What learning algorithm is in-context learning? investigations with linear models. In *International Conference on Learning Representations*, 2023.
- Arora, A., Jurafsky, D., Potts, C., and Goodman, N. D. Bayesian scaling laws for in-context learning, 2024a. URL <https://arxiv.org/abs/2410.16531>.
- Arora, S., Eyuboglu, S., Timalsina, A., Johnson, I., Poli, M., Zou, J., Rudra, A., and Re, C. Zoology: Measuring and improving recall in efficient language models. In *The Twelfth International Conference on Learning Representations*, 2024b. URL <https://openreview.net/forum?id=LY3ukUANKo>.
- Bai, Y., Tu, S., Zhang, J., Peng, H., Wang, X., Lv, X., Cao, S., Xu, J., Hou, L., Dong, Y., et al. Longbench v2: Towards deeper understanding and reasoning on realistic long-context multitasks. In *Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pp. 3639–3664, 2025.
- Beck, M., Pöppel, K., Spanring, M., Auer, A., Prudnikova, O., Kopp, M., Klambauer, G., Brandstetter, J., and Hochreiter, S. xlstm: Extended long shortterm memory. *arXiv preprint arXiv:2405.04517*, 2024.
- Behrouz, A., Zhong, P., and Mirrokni, V. Titans: Learning to memorize at test time. *arXiv preprint arXiv:2501.00663*, 2024.
- Benna, M. K. and Fusi, S. Computational principles of synaptic memory consolidation. *Nature neuroscience*, 19(12):16971706, 2016.
- Bisk, Y., Zellers, R., Gao, J., Choi, Y., et al. Piqa: Reasoning about physical commonsense in natural language. In *Proceedings of the AAAI conference on artificial intelligence*, volume 34, pp. 7432–7439, 2020.
- Blundell, C., Cornebise, J., Kavukcuoglu, K., and Wierstra, D. Weight uncertainty in neural network. In *International conference on machine learning*, pp. 1613–1622. PMLR, 2015.
- Bonnet, D., Cottart, K., Hirtzlin, T., Januel, T., Dalgaty, T., Vianello, E., and Querlioz, D. Bayesian continual learning and forgetting in neural networks. *Nature Communications*, 16(1):9614, 2025.
- Brohan, A., Brown, N., Carbajal, J., Chebotar, Y., Dabis, J., Finn, C., Gopalakrishnan, K., Hausman, K., Herzog, A., Hsu, J., et al. Rt-1: Robotics transformer for real-world control at scale. *arXiv preprint arXiv:2212.06817*, 2022.
- Brown, T., Mann, B., Ryder, N., Subbiah, M., Kaplan, J. D., Dhariwal, P., Neelakantan, A., Shyam, P., Sastry, G., Askell, A., et al. Language models are few-shot learners. *Advances in neural information processing systems*, 33: 1877–1901, 2020.
- Buzzega, P., Boschini, M., Porrello, A., Abati, D., and Calderara, S. Dark experience for general continual learning: a strong, simple baseline. *Advances in neural information processing systems*, 33:15920–15930, 2020.

- Chen, Y., Qian, S., Tang, H., Lai, X., Liu, Z., Han, S., and Jia, J. Longlora: Efficient fine-tuning of long-context large language models. In *International Conference on Learning Representations*, volume 2024, pp. 8220–8238, 2024.
- Cheng, D., Lu, Y., He, L., Zhang, S., Yang, X., Wang, N., and Gao, X. Mambac1: Optimizing selective state space model in null space for continual learning. *arXiv preprint arXiv:2411.15469*, 2024.
- Chowdhery, A., Narang, S., Devlin, J., Bosma, M., Mishra, G., Roberts, A., Barham, P., Chung, H. W., Sutton, C., Gehrmann, S., et al. Palm: Scaling language modeling with pathways. *Journal of Machine Learning Research*, 24(240):1–113, 2023.
- Clark, P., Cowhey, I., Etzioni, O., Khot, T., Sabharwal, A., Schoenick, C., and Tafjord, O. Think you have solved question answering? try arc, the ai2 reasoning challenge. *arXiv preprint arXiv:1803.05457*, 2018.
- Dao, T. and Gu, A. Transformers are ssms: Generalized models and efficient algorithms through structured state space duality. *arXiv preprint arXiv:2405.21060*, 2024.
- Dosovitskiy, A., Beyer, L., Kolesnikov, A., Weissenborn, D., Zhai, X., Unterthiner, T., Dehghani, M., Minderer, M., Heigold, G., Gelly, S., et al. An image is worth 16x16 words: Transformers for image recognition at scale. In *International Conference on Learning Representations*, 2021.
- Falck, F., Wang, Z., and Holmes, C. Is in-context learning in large language models bayesian? a martingale perspective. In *International Conference on Machine Learning*, 2024.
- Gao, T., Wettig, A., Yen, H., and Chen, D. How to train long-context language models (effectively). In *Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pp. 7376–7399, 2025.
- Gu, A. and Dao, T. Mamba: Lineartime sequence modeling with selective state spaces. *arXiv preprint arXiv:2312.00752*, 2023.
- Hahn, M. and Goyal, N. A theory of emergent in-context learning as implicit structure induction. *arXiv preprint arXiv:2303.07971*, 2023.
- Hsieh, C.-P., Sun, S., Krizan, S., Acharya, S., Rekesh, D., Jia, F., Zhang, Y., and Ginsburg, B. Ruler: What’s the real context size of your long-context language models? In *Conference on Language Modeling*, 2024.
- Jumper, J., Evans, R., Pritzel, A., Green, T., Figurnov, M., Ronneberger, O., Tunyasuvunakool, K., Bates, R., Židek, A., Potapenko, A., et al. Highly accurate protein structure prediction with alphafold. *nature*, 596(7873):583–589, 2021.
- Kaushik, P., Gain, A., Kortylewski, A., and Yuille, A. Understanding catastrophic forgetting and remembering in continual learning with optimal relevance mapping. *NeurIPS 2021 Workshop MetaLearn Poster. Available on arXiv preprint arXiv:2102.11343*, 2021.
- Kim, S., Hooper, C., Wattanawong, T., Kang, M., Yan, R., Genc, H., Dinh, G., Huang, Q., Keutzer, K., Mahoney, M. W., et al. Full stack optimization of transformer inference: a survey. *arXiv preprint arXiv:2302.14017*, 2023.
- Kirkpatrick, J., Pascanu, R., Rabinowitz, N., Veness, J., Desjardins, G., Rusu, A. A., Milan, K., Quan, J., Ramalho, T., Grabska-Barwinska, A., et al. Overcoming catastrophic forgetting in neural networks. *Proceedings of the national academy of sciences*, 114(13):3521–3526, 2017a.
- Kirkpatrick, J., Pascanu, R., Rabinowitz, N., Veness, J., Desjardins, G., Rusu, A. A., Milan, K., Quan, J., Ramalho, T., GrabskaBarwinska, A., et al. Overcoming catastrophic forgetting in neural networks. *Proceedings of the national academy of sciences*, pp. 201611835, 2017b.
- Kudithipudi, D., Daram, A., Ziyarah, A. M., Zohora, F. T., Aimone, J. B., YanguasGil, A., Soures, N., Neftci, E., Mattina, M., Lomonaco, V., et al. Design principles for lifelong learning ai accelerators. *Nature Electronics*, pp. 116, 2023.
- Liu, B., Wang, R., Wu, L., Feng, Y., Stone, P., and Liu, Q. Longhorn: State space models are amortized online learners. In *International Conference on Learning Representations*, volume 2025, pp. 95419–95434, 2025.
- Lopez-Paz, D. and Ranzato, M. Gradient episodic memory for continual learning. *Advances in neural information processing systems*, 30, 2017.
- McClelland, J. L., McNaughton, B. L., and O’Reilly, R. C. Why there are complementary learning systems in the hippocampus and neocortex: insights from the successes and failures of connectionist models of learning and memory. *Psychological review*, 102(3):419, 1995.
- McCloskey, M. and Cohen, N. J. Catastrophic interference in connectionist networks: The sequential learning problem. In *Psychology of learning and motivation*, volume 24, pp. 109165. Elsevier, 1989.
- Paperno, D., Kruszewski, G., Lazaridou, A., Pham, Q. N., Bernardi, R., Pezzelle, S., Baroni, M., Boleda, G., and Fernández, R. The lambada dataset: Word prediction

- p>
 requiring a broad discourse context. In
- Proceedings of the 54th Annual Meeting of the Association for Computational Linguistics*
- , pp. 1525–1534, 2016.
- Rusu, A. A., Rabinowitz, N. C., Desjardins, G., Soyer, H., Kirkpatrick, J., Kavukcuoglu, K., Pascanu, R., and Hassel, R. Progressive neural networks. *arXiv preprint arXiv:1606.04671*, 2016.
- Sakaguchi, K., Le Bras, R., Bhagavatula, C., and Choi, Y. Winogrande: An adversarial winograd schema challenge at scale. *Communications of the ACM*, 64(9):99–106, 2021.
- Sap, M., Rashkin, H., Chen, D., LeBras, R., and Choi, Y. Socialiqa: Commonsense reasoning about social interactions. *arXiv preprint arXiv:1904.09728*, 2019.
- Schlag, I., Irie, K., and Schmidhuber, J. Linear transformers are secretly fast weight programmers. In *International Conference on Machine Learning*, pp. 9359366. PMLR, 2021.
- Schwaller, P., Laino, T., Gaudin, T., Bolgar, P., Hunter, C. A., Bekas, C., and Lee, A. A. Molecular transformer: a model for uncertainty-calibrated chemical reaction prediction. *ACS central science*, 5(9):1572–1583, 2019.
- Sun, Y., Li, X., Dalal, K., Hsu, C., Koyejo, S., Guestrin, C., Wang, X., Hashimoto, T., and Chen, X. Learning to (learn at test time). *arXiv preprint arXiv:2310.13807*, 2023.
- Sun, Y., Fujisawa, I., Juliani, A., Sakuma, J., and Kanai, R. Remembering transformer for continual learning. *arXiv preprint arXiv:2404.07518*, 2024.
- Thengane, V., Khan, S., Hayat, M., and Khan, F. Clip model is an efficient continual learner, 2022.
- Vaswani, A., Shazeer, N., Parmar, N., Uszkoreit, J., Jones, L., Gomez, A. N., Kaiser, Ł., and Polosukhin, I. Attention is all you need. In *Advances in neural information processing systems*, pp. 59986008, 2017.
- Von Oswald, J., Niklasson, E., Randazzo, E., Sacramento, J., Mordvintsev, A., Zhmoginov, A., and Vladymyrov, M. Transformers learn in-context by gradient descent. In *International Conference on Machine Learning*, pp. 35151–35174. PMLR, 2023a.
- Von Oswald, J., Schlegel, M., Meulemans, A., Kobayashi, S., Niklasson, E., Zucchet, N., Scherrer, N., Miller, N., Sandler, M., Vladymyrov, M., et al. Uncovering mesa-optimization algorithms in transformers. *arXiv preprint arXiv:2309.05858*, 2023b.
- Von Oswald, J., Scherrer, N., Kobayashi, S., Versari, L., Yang, S., Schlegel, M., Maile, K., Schimpf, Y., Sieberling, O., Meulemans, A., et al. Mesanet: Sequence modeling by locally optimal test-time training. *arXiv preprint arXiv:2506.05233*, 2025.
- Wang, H., Lu, H., Yao, L., and Gong, D. Self-expansion of pre-trained models with mixture of adapters for continual learning. *arXiv preprint arXiv:2403.18886*, 2024.
- Wies, N., Levine, Y., and Shashua, A. The learnability of in-context learning. *Advances in Neural Information Processing Systems*, 36:36637–36651, 2023.
- Xie, S. M., Raghunathan, A., Liang, P., and Ma, T. An explanation of in-context learning as implicit bayesian inference. *arXiv preprint arXiv:2111.02080*, 2021.
- Yang, S., Wang, B., Shen, Y., Panda, R., and Kim, Y. Gated linear attention transformers with hardware-efficient training. In *International Conference on Machine Learning*, 2024.
- Yang, S., Kautz, J., and Hatamizadeh, A. Gated delta networks: Improving mamba2 with delta rule. In *International Conference on Learning Representations*, volume 2025, pp. 29687–29707, 2025.
- Zellers, R., Holtzman, A., Bisk, Y., Farhadi, A., and Choi, Y. HellaSwag: Can a machine really finish your sentence? In Korhonen, A., Traum, D., and Màrquez, L. (eds.), *Proceedings of the 57th Annual Meeting of the Association for Computational Linguistics*, pp. 4791–4800, Florence, Italy, July 2019. Association for Computational Linguistics. doi: 10.18653/v1/P19-1472. URL <https://aclanthology.org/P19-1472/>.
- Zenke, F., Poole, B., and Ganguli, S. Continual learning through synaptic intelligence. In Precup, D. and Teh, Y. W. (eds.), *Proceedings of the 34th International Conference on Machine Learning*, volume 70 of *Proceedings of Machine Learning Research*, pp. 39873995. PMLR, 0611 Aug 2017. URL <https://proceedings.mlr.press/v70/zenke17a.html>.
- Zeno, C., Golan, I., Hoffer, E., and Soudry, D. Task-agnostic continual learning using online variational bayes with fixed-point updates. *Neural Computation*, 33(11):3139–3177, 2021.
- Zhang, Y. and Yang, S. Flame: Flash language modeling made easy, January 2025. URL <https://github.com/fla-org/flame>.
- Zhang, Y., Zhang, F., Yang, Z., and Wang, Z. What and how does in-context learning learn? bayesian model averaging, parameterization, and generalization. *arXiv preprint arXiv:2305.19420*, 2023.

Zhang, Y., Lin, Z., Sun, Y., Yin, F., and Fritsche, C.
Regularization-based efficient continual learning in deep
state-space models. In *2024 27th International Confer-
ence on Information Fusion (FUSION)*, pp. 1–8. IEEE,
2024.

A. Appendices

As described in the main text, Palimpsest-M builds upon Mamba2, while Palimpsest-D is based on Deltanets. These two models utilize distinct gated linear attention layers and separate backbones, allowing our metaplasticity ablation studies to evaluate two highly different points in the architectural design space.

The most striking difference is that Palimpsest-M lacks a gated MLP and relies solely on its gated linear attention layer projections for channel mixing, which is the standard configuration for Mamba2. In contrast, Palimpsest-D follows Deltanet principles by incorporating a gated MLP after the linear attention stage.

Furthermore, the models handle input gating differently: Palimpsest-M utilizes d_t (a term already present in the forgetting mechanism) for input gating, whereas Palimpsest-D employs a dedicated parameter b_t . This allows Palimpsest-D to decorrelate forgetting and input integration. Additional distinctions include the order of the output gating and normalization at the end of the layer, and the fact that Palimpsest-M does not normalize the queries (q_t) and keys (k_t).

A.1. Palimpsest Model Variants

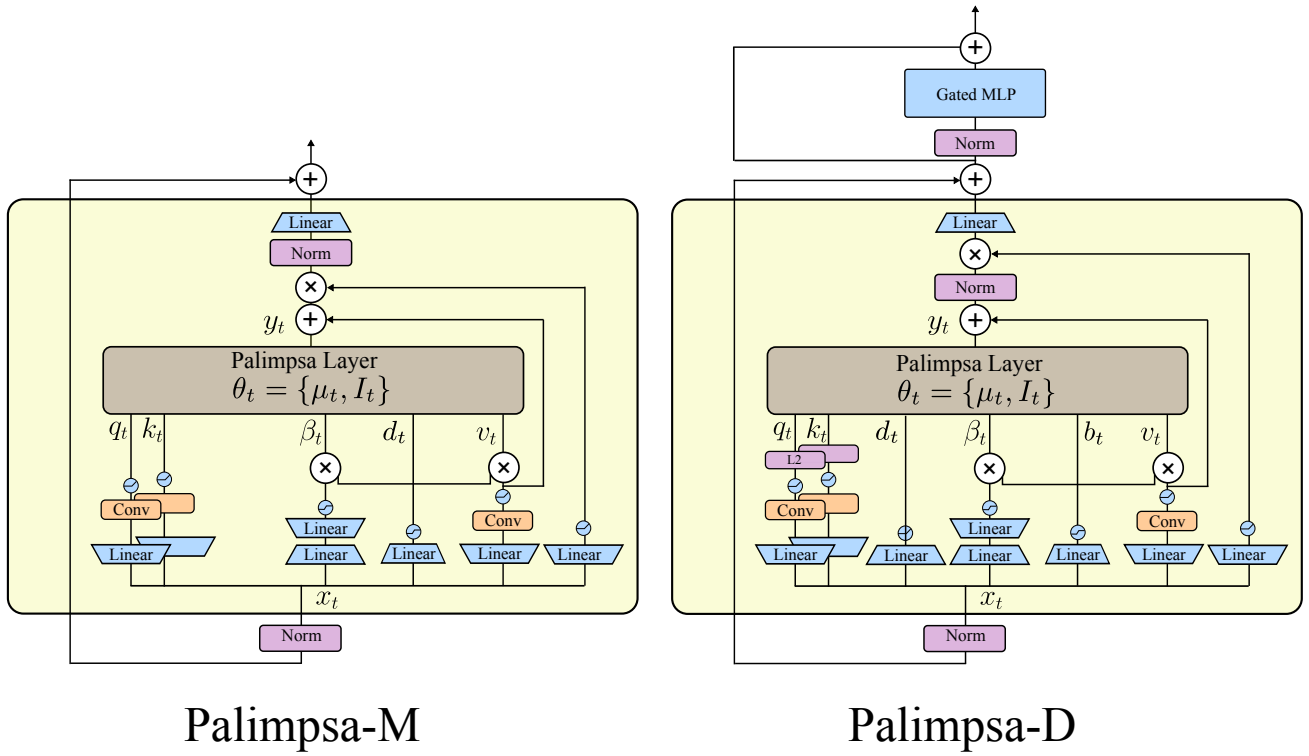


Figure 5. Illustration of the Palimpsest-M and Palimpsest-D architectures. While Palimpsest-M adopts the Mamba2 configuration by relying on the attention layer for channel mixing, Palimpsest-D incorporates an explicit gated MLP. Additionally, Palimpsest-D introduces a dedicated b_t parameter to decorrelate input integration from the forgetting dynamics dictated by d_t .

A.2. Bayesian Forgetting

Following the work of (Bonnet et al., 2025), we introduce a forgetting mechanism by considering a truncated posterior, which contains the last N data points. The posterior over parameters $\mathbf{S}$ given data $\mathbf{d}_{t-N:t}$ is:

$$p(\mathbf{S}|\mathbf{d}_{t-N:t}) = \frac{p(\mathbf{d}_{t-N:t}|\mathbf{S})p(\mathbf{S})}{p(\mathbf{d}_{t-N:t})} = \frac{p(\mathbf{d}_t|\mathbf{S})p(\mathbf{S}|\mathbf{d}_{t-N:t-1})}{p(\mathbf{d}_t|\mathbf{d}_{t-N:t-1})}. \quad (6)$$

This can be expressed as a recursive update for a sliding window of size N . To move from the posterior over $\mathbf{d}_{t-N:t-1}$ to the one over $\mathbf{d}_{t-N+1:t}$, we incorporate the new data point $\mathbf{d}_t$ and remove the oldest one, $\mathbf{d}_{t-N}$:

$$p(\mathbf{S}|\mathbf{d}_{t-N+1:t}) \propto \underbrace{p(\mathbf{d}_t|\mathbf{S})p(\mathbf{S}|\mathbf{d}_{t-N:t-1})}_{\text{Learning}} \cdot \underbrace{\frac{1}{p(\mathbf{d}_{t-N}|\mathbf{S})}}_{\text{Forgetting}}. \quad (7)$$

In principle, one may not have access at time t to the likelihood of the oldest data point, $p(\mathbf{d}_{t-N}|\mathbf{S})$. As a proxy, one can use the geometric mean of the likelihoods over the previous window, $\mathbf{d}_{t-N:t-1}$:

$$p(\mathbf{d}_{t-N}, \dots, \mathbf{d}_{t-1}|\mathbf{S})^{\frac{1}{N}} \propto \left[\frac{p(\mathbf{S}|\mathbf{d}_{t-N:t-1})}{p(\mathbf{S})} \right]^{\frac{1}{N}}. \quad (8)$$

This is made possible by approximating $p(\mathbf{S}|\mathbf{d}_{t-N:t-1})$, by the current variational truncated posterior $q_{\theta_{t-1}}(\mathbf{S})$.

In simple terms, instead of completely discarding a single data point, this approach suggests we can discard a fraction of the joint likelihood of past data. This leads to a ‘weighted’ posterior where older data points have been discounted more heavily over time, keeping the total ‘weight’ of the posterior roughly constant (when $t \gg N$).

For Palimpsest, the forgetting needs to be input-dependent, similar to other gated recurrent models. Therefore, at each time step t , we discount the influence of all previous data by a factor $\frac{1}{N_t} \in [0, 1]$. The resulting weighted posterior at time t , denoted $p_w(\mathbf{S}|\mathbf{d}_{1:t})$, is defined recursively as:

$$p_w(\mathbf{S}|\mathbf{d}_{1:t}) \propto \underbrace{p(\mathbf{d}_t|\mathbf{S})p_w(\mathbf{S}|\mathbf{d}_{1:t-1})}_{\text{Learning}} \cdot \underbrace{\left(\frac{p_w(\mathbf{S}|\mathbf{d}_{1:t-1})}{p(\mathbf{S})} \right)^{-\frac{1}{N_t}}}_{\text{Forgetting}}. \quad (9)$$

This is the target probability distribution that the variational distribution in Palimpsest approximates. Notably, this form of weighted posterior is the starting point for the general Bayesian framework for gated recurrent models.

A.3. Variational Free Energy of the Weighted Posterior

We minimize the Kullback–Leibler (KL) divergence between the variational distribution $q_{\theta}(\mathbf{S}_i)$ and the weighted posterior $p_w(\mathbf{S}_i|\mathbf{d}_{1:t})$ by finding the optimal parameters $\theta_{t,i}$:

$$\theta_{t,i} = \underset{\theta}{\operatorname{argmin}} D_{\text{KL}} [q_{\theta}(\mathbf{S}_i) \| p_w(\mathbf{S}_i|\mathbf{d}_{1:t})], \quad i = 1, \dots, d_v.$$

Using the definition of the KL divergence, the recursive update from the previous section (Equation 9), and assuming that the previous posterior is well-approximated by our variational distribution, $p_w(\mathbf{S}_i|\mathbf{d}_{1:t-1}) \approx q_{\theta_{t-1,i}}(\mathbf{S}_i)$, we can show that this is equivalent to minimizing the variational free energy $\mathcal{F}_{t,i}$:

$$\mathcal{F}_{t,i} = D_{\text{KL}} [q_{\theta_{t,i}}(\mathbf{S}_i) \| q_{\theta_{t-1,i}}(\mathbf{S}_i)] - \mathbb{E}_{q_{\theta_{t,i}}(\mathbf{S}_i)} \left[\log p(\mathbf{d}_t|\mathbf{S}_i) - \frac{1}{N_t} \log \frac{q_{\theta_{t-1,i}}(\mathbf{S}_i)}{p(\mathbf{S}_i)} \right].$$

Now, using the standard formulas for Gaussian distributions (see A.7 for a refresher) and considering only the terms that depend on the variational mean μ_i , we obtain the free energy expression discussed in the main text:

$$\mathcal{F}_{t,i}(\mu_i) = \underbrace{\frac{\beta_{t,i}}{2} \|\mu_i^T \mathbf{k}_t - v_{t,i}\|^2}_{\text{plasticity}} + \underbrace{\frac{1}{N_t} \frac{\|\mu_i\|^2}{2\sigma_{\text{prior}}^2}}_{\text{forgetting}} + \underbrace{\frac{1}{2} \left(1 - \frac{1}{N_t} \right) (\mu_{t-1,i} - \mu_i)^T \Sigma_{t-1,i}^{-1} (\mu_{t-1,i} - \mu_i)}_{\text{stability}}.$$

For Palimpsest we are also interested in the terms that depend on $\Sigma_{t,i}$. First, let’s derive the gradient with respect to all variational parameters.

Calculating the Gradient of $\mathcal{F}_{t,i}$: For readability, we first define the cost term $\mathcal{C}_{t,i}$ as the expected negative log-likelihood:

$$\mathcal{C}_{t,i} := -\mathbb{E}_{q_{\theta_i}(\mathbf{S}_i)} [\log p(v_{t,i}|\mathbf{k}_t, \beta_{t,i}, \mathbf{S}_i)], \quad (10)$$

To compute the partial derivatives of $\mathcal{C}_{t,i}$, we use the reparameterization trick: $\mathbf{S}_i = \mu_i + \mathbf{A}_i \epsilon_i$, and $\mathbf{A}_i \mathbf{A}_i^T = \Sigma_{t,i}$, where

$\epsilon_i \sim \mathcal{N}(0, \mathbb{I})$. The cost function can now be written as follows:

$$\mathcal{C}_{t,i} = \mathbb{E}_{\epsilon_i} [\mathcal{L}_i(\mathbf{S}_i)] \quad (11)$$

$$\frac{\partial \mathcal{C}_i}{\partial \boldsymbol{\mu}_i} = \mathbb{E}_{\epsilon_i} \left[\frac{\partial \mathcal{L}_i(\mathbf{S}_i)}{\partial \mathbf{S}_i} \right] \quad \frac{\partial \mathcal{C}_i}{\partial \mathbf{A}_i} = \mathbb{E}_{\epsilon_i} \left[\frac{\partial \mathcal{L}_i(\mathbf{S}_i)}{\partial \mathbf{S}_i} \epsilon_i^\top \right]. \quad (12)$$

Generally, these gradients would be estimated using Monte Carlo sampling. However, in Palimpse, ICL amounts to a linear regression problem so the gradients can be calculated analytically. Substituting the derivative of the linear regression log-likelihood gives:

$$\frac{\partial \mathcal{C}_i}{\partial \boldsymbol{\mu}_i} = \mathbb{E}_{\epsilon_i} [\beta_{t,i} ((\boldsymbol{\mu}_i + \mathbf{A}_i \epsilon_i)^\top \mathbf{k}_t - v_{t,i}) \mathbf{k}_t] \quad \frac{\partial \mathcal{C}_i}{\partial \mathbf{A}_i} = \mathbb{E}_{\epsilon_i} [\beta_{t,i} ((\boldsymbol{\mu}_i + \mathbf{A}_i \epsilon_i)^\top \mathbf{k}_t - v_{t,i}) \mathbf{k}_t \epsilon_i^\top]. \quad (13)$$

Knowing that $\mathbb{E}_{\epsilon_i} [\epsilon_i] = \vec{0}$ and $\mathbb{E}_{\epsilon_i} [\epsilon_i \epsilon_i^\top] = \mathbb{I}$ (the identity matrix), we can resolve the expectations to get the final analytical gradients of the cost term:

$$\frac{\partial \mathcal{C}_i}{\partial \boldsymbol{\mu}_i} = \beta_{t,i} (\mathbf{k}_t \mathbf{k}_t^\top \boldsymbol{\mu}_i - v_{t,i} \mathbf{k}_t) \quad \frac{\partial \mathcal{C}_i}{\partial \mathbf{A}_i} = \beta_{t,i} \mathbf{k}_t \mathbf{k}_t^\top \mathbf{A}_i. \quad (14)$$

Finally, we add the gradients from the other terms in the free energy objective (the KL-divergence and forgetting terms) to obtain the full gradients of $\mathcal{F}_{t,i}$:

$$\frac{\partial \mathcal{F}_{t,i}}{\partial \boldsymbol{\mu}_i} = \left(1 - \frac{1}{N_t}\right) \boldsymbol{\Sigma}_{t-1,i}^{-1} (\boldsymbol{\mu}_i - \boldsymbol{\mu}_{t-1,i}) + \frac{1}{N_t} \boldsymbol{\mu}_i + \beta_{t,i} (\mathbf{k}_t \mathbf{k}_t^\top \boldsymbol{\mu}_i - v_{t,i} \mathbf{k}_t), \quad (15)$$

$$\frac{\partial \mathcal{F}_{t,i}}{\partial \mathbf{A}_i} = \left[\left(1 - \frac{1}{N_t}\right) \boldsymbol{\Sigma}_{t-1,i}^{-1} + \frac{1}{N_t} \mathbf{I}_{prior} \right] \mathbf{A}_i - (\mathbf{A}_i^{-1})^\top + \beta_{t,i} \mathbf{k}_t \mathbf{k}_t^\top \mathbf{A}_i. \quad (16)$$

Closed-Form Solution of $\mathcal{F}_{t,i}$: To find the optimal variational parameters, we set the gradients of the free energy $\mathcal{F}_{t,i}$ to zero and solve. First, to find the optimal covariance matrix $\boldsymbol{\Sigma}_{t,i}$, we solve for $\mathbf{A}_{t,i}$ in the equation:

$$\frac{\partial \mathcal{F}_{t,i}}{\partial \mathbf{A}_{t,i}} = \mathbf{0}. \quad (17)$$

Setting the gradient expression from the previous section to zero gives:

$$\left[\left(1 - \frac{1}{N_t}\right) \boldsymbol{\Sigma}_{t-1,i}^{-1} + \frac{1}{N_t} \mathbf{I}_{prior} + \beta_{t,i} \mathbf{k}_t \mathbf{k}_t^\top \right] \mathbf{A}_{t,i} - (\mathbf{A}_{t,i}^{-1})^\top = \mathbf{0}. \quad (18)$$

To solve for the full covariance matrix $\boldsymbol{\Sigma}_{t,i} = \mathbf{A}_{t,i} \mathbf{A}_{t,i}^\top$, we can multiply on the right by $\mathbf{A}_{t,i}^\top$:

$$\left[\left(1 - \frac{1}{N_t}\right) \boldsymbol{\Sigma}_{t-1,i}^{-1} + \frac{1}{N_t} \mathbf{I}_{prior} + \beta_{t,i} \mathbf{k}_t \mathbf{k}_t^\top \right] \boldsymbol{\Sigma}_{t,i} - \mathbf{I} = \mathbf{0}. \quad (19)$$

Rearranging the terms, we find that the new precision matrix (the inverse covariance) is a sum of the discounted old precision, the prior precision, and the new data term:

$$\boldsymbol{\Sigma}_{t,i}^{-1} = \left(1 - \frac{1}{N_t}\right) \boldsymbol{\Sigma}_{t-1,i}^{-1} + \frac{1}{N_t} \mathbf{I}_{prior} + \beta_{t,i} \mathbf{k}_t \mathbf{k}_t^\top. \quad (20)$$

The closed-form update for the covariance is the inverse of this expression:

$$\boxed{\boldsymbol{\Sigma}_{t,i} = \left[\left(1 - \frac{1}{N_t}\right) \boldsymbol{\Sigma}_{t-1,i}^{-1} + \frac{1}{N_t} \mathbf{I}_{prior} + \beta_{t,i} \mathbf{k}_t \mathbf{k}_t^\top \right]^{-1}} \quad (21)$$

As discussed in the main text, the major challenge in this analytical approach is to compute the $d_k \times d_k$ matrix inverse at each time step. In the case without forgetting ($N \rightarrow \infty$), the prior term vanishes, and the precision matrix update is a pure

rank-1 addition: $\Sigma_{t,i}^{-1} = \Sigma_{t-1,i}^{-1} + \beta_{t,i} \mathbf{k}_t \mathbf{k}_t^\top$. In this regime, the covariance matrix can be recursively computed without direct matrix inversion using the standard Sherman-Morrison formula:

$$\Sigma_{t,i} = \Sigma_{t-1,i} - \frac{\beta_{t,i}}{1 + \beta_{t,i} \mathbf{k}_t^\top \Sigma_{t-1,i} \mathbf{k}_t} \Sigma_{t-1,i} \mathbf{k}_t \mathbf{k}_t^\top \Sigma_{t-1,i}. \quad (22)$$

If we reintroduce forgetting ($N_t < \infty$) the precision update consists of rank- d_k update, $\mathbf{I}_{prior}$ being typically the identity matrix here. The full covariance matrix cannot, therefore, be computed exactly using Sherman-Morrison formula.

A.4. Derivation of Mesanet

In the case without forgetting ($N \rightarrow \infty$), (Von Oswald et al., 2023b) use the Sherman-Morrison formula to compute it recursively, but this scaled poorly. Another approach is to use a parallel conjugate gradient method to solve the associated linear systems, as done in Mesanet (Von Oswald et al., 2025). To connect this with the notation in Mesanet (Von Oswald et al., 2025), our precision matrix Σ_t^{-1} corresponds to their $\mathbf{H}_t + \mathbf{\Lambda}$, and our precision-weighted mean $\mu_t \Sigma_t^{-1}$ corresponds to their $\mathbf{G}_t$. In their work, the precision matrix is identical for all rows i (i.e., $\Sigma_{t,i}^{-1} = \Sigma_t^{-1}$) because their β_t is a scalar.

Similarly, we find the optimal mean $\mu_{t,i}$ by setting its corresponding gradient to zero:

$$\frac{\partial \mathcal{F}_{t,i}}{\partial \mu_{t,i}} = \vec{0}. \quad (23)$$

$$\left(1 - \frac{1}{N_t}\right) \Sigma_{t-1}^{-1} (\mu_{t,i} - \mu_{t-1,i}) + \frac{1}{N_t} \mathbf{I}_{prior} \mu_{t,i} + \beta_t (\mathbf{k}_t \mathbf{k}_t^\top \mu_{t,i} - v_{t,i} \mathbf{k}_t) = \vec{0}. \quad (24)$$

Grouping the terms with $\mu_{t,i}$ yields:

$$\left[\left(1 - \frac{1}{N_t}\right) \Sigma_{t-1}^{-1} + \frac{1}{N_t} \mathbf{I}_{prior} + \beta_t \mathbf{k}_t \mathbf{k}_t^\top\right] \mu_{t,i} = \left(1 - \frac{1}{N_t}\right) \Sigma_{t-1}^{-1} \mu_{t-1,i} + \beta_t v_{t,i} \mathbf{k}_t. \quad (25)$$

Recognizing the term in brackets as the new precision Σ_t^{-1} , we arrive at the solution for the row-mean:

$$\mu_{t,i} = \Sigma_t \left[\left(1 - \frac{1}{N_t}\right) \Sigma_{t-1}^{-1} \mu_{t-1,i} + \beta_t v_{t,i} \mathbf{k}_t \right] \quad (26)$$

By defining the forgetting factor $\alpha_t = \left(1 - \frac{1}{N_t}\right)$ and stacking the transposed row vectors $\mu_{t,i}^\top$ into the full state matrix $\mu_t \in \mathbb{R}^{d_v \times d_k}$, we can write the final update in matrix form. Since the precision and covariance matrices are symmetric ($\Sigma^\top = \Sigma$), transposing the equation shifts these matrices to the right, ensuring the dimensions properly align:

$$\mu_t = [\alpha_t \mu_{t-1} \Sigma_{t-1}^{-1} + \beta_t v_t \mathbf{k}_t^\top] \Sigma_t \quad (27)$$

A.5. Derivation of Palimpsa

In Palimpsa, we aim to solve the update equations with a vector input gating $\beta_t \in \mathbb{R}^{d_v}$, where each row of states has its own rate. This change prevents a simple closed-form solution for the matrix inverse. To make the problem tractable and computationally efficient, we introduce a diagonal approximation for the precision matrix. By assuming the precision matrices are diagonal, $\Sigma_{t,i}^{-1} = \text{diag}(\mathbf{I}_{t,i})$, we can apply this approximation to the closed-form solutions from the previous section. This yields element-wise update rules for the diagonal precision vector $\mathbf{I}_{t,i}$ and the mean vector $\mu_{t,i}$ for each row i :

$$\mathbf{I}_{t,i} = \left(1 - \frac{1}{N_t}\right) \mathbf{I}_{t-1,i} + \frac{1}{N_t} \mathbf{I}_{prior} + \beta_{t,i} \mathbf{k}_t^2 \quad (28)$$

$$\mu_{t,i} = \left(1 - \frac{1}{N_t}\right) \frac{\mathbf{I}_{t-1,i}}{\mathbf{I}_{t,i}} \odot \mu_{t-1,i} + \frac{1}{\mathbf{I}_{t,i}} \odot (\beta_{t,i} v_{t,i} \mathbf{k}_t) \quad (29)$$

where $\mathbf{k}_t^2$ denotes the element-wise square of $\mathbf{k}_t$, and all divisions are element-wise. Note that this is a stronger approximation than a standard mean-field (diagonal covariance) assumption because we derived the fully-coupled solution first and only then discarded the off-diagonal terms. This approach is similar to the diagonal approximation used in Longhorn (Liu et al., 2025) and is equivalent to treating each parameter (or "synapse") as an independent Gaussian distribution.

From here, by defining the forgetting factor $\alpha_t = (1 - \frac{1}{N_t})$ and stacking the row vectors into matrices, we can write the final updates for Palimpsest in matrix form:

$$\mathbf{I}_t = \alpha_t \mathbf{I}_{t-1} + (1 - \alpha_t) \mathbf{I}_{prior} + \beta_t \otimes \mathbf{k}_t^2 \quad (30)$$

$$\boldsymbol{\mu}_t = \alpha_t \frac{\mathbf{I}_{t-1}}{\mathbf{I}_t} \odot \boldsymbol{\mu}_{t-1} + \frac{1}{\mathbf{I}_t} \odot [(\beta_t \odot \mathbf{v}_t) \otimes \mathbf{k}_t] \quad (31)$$

A.6. Derivation of Deltanet and Gated Deltanet

Gated Deltanet can be derived similarly by suppressing the forgetting for the standard Deltanet. Starting from the free energy equation and taking $\beta_t \in \mathbb{R}$:

$$\mathcal{F}_{t,i}(\boldsymbol{\mu}_i) = \underbrace{\frac{\beta_t}{2} \|\boldsymbol{\mu}_i^T \mathbf{k}_t - v_{t,i}\|^2}_{\text{plasticity}} + \underbrace{\frac{1}{N_t} \frac{\|\boldsymbol{\mu}_i\|^2}{2\sigma_{prior}^2}}_{\text{forgetting}} + \underbrace{\frac{1}{2} \left(1 - \frac{1}{N_t}\right) (\boldsymbol{\mu}_{t-1,i} - \boldsymbol{\mu}_i)^T \boldsymbol{\Sigma}_{t-1,i}^{-1} (\boldsymbol{\mu}_{t-1,i} - \boldsymbol{\mu}_i)}_{\text{stability}}.$$

with the simplifications $\boldsymbol{\Sigma}_{t-1,i}^{-1} = \mathbb{I}_{d_k}$ and $I_{prior} = 1$. If we consider the plasticity term of the loss to be linear around $\boldsymbol{\mu}_{t-1,i}$ (a first-order approximation), we assume that:

$$\beta_t (\mathbf{k}_t \mathbf{k}_t^\top \boldsymbol{\mu}_i - v_{t,i} \mathbf{k}_t) \approx \beta_t (\mathbf{k}_t \mathbf{k}_t^\top \boldsymbol{\mu}_{t-1,i} - v_{t,i} \mathbf{k}_t).$$

Then, setting the full gradient to zero, $\frac{\partial \mathcal{F}_{t,i}}{\partial \boldsymbol{\mu}_i} = \vec{0}$, is given by:

$$\left(1 - \frac{1}{N_t}\right) \mathbb{I}_{d_k} (\boldsymbol{\mu}_i - \boldsymbol{\mu}_{t-1,i}) + \frac{1}{N_t} \boldsymbol{\mu}_i + \beta_t (\mathbf{k}_t \mathbf{k}_t^\top \boldsymbol{\mu}_{t-1,i} - v_{t,i} \mathbf{k}_t) = \vec{0}. \quad (32)$$

Solving for $\boldsymbol{\mu}_i$ yields:

$$\boldsymbol{\mu}_i = \left(1 - \frac{1}{N_t}\right) \boldsymbol{\mu}_{t-1,i} - \beta_t (\mathbf{k}_t \mathbf{k}_t^\top \boldsymbol{\mu}_{t-1,i} - v_{t,i} \mathbf{k}_t). \quad (33)$$

From there, by defining $\alpha_t = (1 - \frac{1}{N_t})$, we can write the final solution in matrix form as:

$$\boldsymbol{\mu}_t = \boldsymbol{\mu}_{t-1} (\alpha_t \mathbb{I}_{d_k} - \beta_t \mathbf{k}_t \mathbf{k}_t^\top) + \beta_t \mathbf{v}_t \mathbf{k}_t^\top. \quad (34)$$

This is equivalent to Gated Deltanet with the right reparametrization, and the same as the standard Deltanet when $N \rightarrow \infty$, i.e., $\alpha_t \rightarrow 1$.

A.7. Gaussian Cheat Sheet

Entropy The entropy of a multivariate Gaussian distribution $q_{\theta_1}(\mathbf{S}_i) = \mathcal{N}(\boldsymbol{\mu}_1, \boldsymbol{\Sigma}_1)$ of dimension d_k is given by:

$$H(q_{\theta_1}) = -\mathbb{E}_{q_{\theta_1}(\mathbf{S}_i)} [\log q_{\theta_1}(\mathbf{S}_i)]$$

$$H(q_{\theta_1}) = \frac{d_k}{2} \log(2\pi e) + \frac{1}{2} \log \det(\boldsymbol{\Sigma}_1)$$

KL Divergence The KL divergence between two multivariate Gaussian distributions $q_{\theta_1}(\mathbf{S}_i) = \mathcal{N}(\boldsymbol{\mu}_1, \boldsymbol{\Sigma}_1)$ and $q_{\theta_2}(\mathbf{S}_i) = \mathcal{N}(\boldsymbol{\mu}_2, \boldsymbol{\Sigma}_2)$ is given by:

$$D_{\text{KL}}[q_{\theta_1}(\mathbf{S}_i) || q_{\theta_2}(\mathbf{S}_i)] = \mathbb{E}_{q_{\theta_1}(\mathbf{S}_i)} \left[\log \frac{q_{\theta_1}(\mathbf{S}_i)}{q_{\theta_2}(\mathbf{S}_i)} \right]$$

$$D_{\text{KL}}[q_{\theta_1}(\mathbf{S}_i) || q_{\theta_2}(\mathbf{S}_i)] = \frac{1}{2} \left[\text{tr}(\boldsymbol{\Sigma}_2^{-1} \boldsymbol{\Sigma}_1) + (\boldsymbol{\mu}_2 - \boldsymbol{\mu}_1)^T \boldsymbol{\Sigma}_2^{-1} (\boldsymbol{\mu}_2 - \boldsymbol{\mu}_1) - d_k + \log \frac{\det \boldsymbol{\Sigma}_2}{\det \boldsymbol{\Sigma}_1} \right]$$

Cross-Entropy The cross-entropy between two multivariate Gaussian distributions $q_{\theta_1}(\mathbf{S}_i) = \mathcal{N}(\boldsymbol{\mu}_1, \boldsymbol{\Sigma}_1)$ and $q_{\theta_2}(\mathbf{S}_i) = \mathcal{N}(\boldsymbol{\mu}_2, \boldsymbol{\Sigma}_2)$ is given by:

$$H(q_{\theta_1}, q_{\theta_2}) = -\mathbb{E}_{q_{\theta_1}(\mathbf{S}_i)} [\log q_{\theta_2}(\mathbf{S}_i)]$$

It can be found using the relation:

$$H(q_{\theta_1}, q_{\theta_2}) = H(q_{\theta_1}) + D_{\text{KL}}[q_{\theta_1}(\mathbf{S}_i) \parallel q_{\theta_2}(\mathbf{S}_i)]$$

The final expression is:

$$H(q_{\theta_1}, q_{\theta_2}) = \frac{1}{2} [\text{tr}(\Sigma_2^{-1} \Sigma_1) + (\boldsymbol{\mu}_2 - \boldsymbol{\mu}_1)^T \Sigma_2^{-1} (\boldsymbol{\mu}_2 - \boldsymbol{\mu}_1) + d_k \log(2\pi) + \log \det \Sigma_2]$$

A.8. MQAR Experiments

For the Multi-Query Associative Recall (MQAR) task, we employ a progressive curriculum learning strategy across four stages of increasing complexity. Models are trained sequentially on sequence lengths $L \in \{128, 256, 512, 1024\}$ with a corresponding number of key-value pairs $N_{kv} \in \{32, 64, 128, 256\}$. We perform a grid search over learning rates and execute each configuration across 8 independent random seeds to ensure statistical robustness.

The recurrence dynamics are initialized to favor long-term memory retention. Specifically, the decay parameters $\mathbf{A}$ are initialized using a linear ramp $\mathcal{U}(0.01, 0.16)$ across heads. The d_t bias is initialized via a linear ramp in log-space between 0.001 and 0.1. Full hyperparameters are detailed in Table 4.

Table 4. Hyperparameters for the MQAR curriculum experiments. Initializations for $\mathbf{A}$ and d_t are configured to favor long-term memory retention.

Hyperparameter	Value / Search Space
Vocabulary size	8,192
Embedding dimension (d_{model})	128
Number of layers	2
Number of heads (n_{heads})	8
State dimension (d_{state})	16
Value expansion (E_v)	2
Key expansion (E_k)	1
β step rank	$d_{model}/16 = 8$
$\mathbf{A}$ initialization	linspace(0.01, 0.16)
Δt initialization	logspace(10^{-3} , 10^{-1})
Curriculum stages (L, N_{kv})	(128, 32), (256, 64), (512, 128), (1024, 256)
Epochs per stage	20
Batch size	128 (for $L \leq 512$), 64 (for $L = 1024$)
Optimizer	AdamW
Learning rate	[1e-3, 2.15e-3, 4.64e-3, 1e-2]
Random seeds	{1, 2, 3, 4, 5, 6, 7, 8}

A.9. Language Modelling Experiments

We evaluate Palimpse across two primary model scales: 170M and 760M parameters. For all models, word embeddings are tied with the language modeling head. The decay parameters $\mathbf{A}$ are initialized by sampling from a uniform distribution $\mathcal{U}(0, 16)$. For discretization, the Δ bias is initialized via a log-space ramp between 0.001 and 0.1.

Convolutional and linear layers are initialized with a Gaussian distribution ($\sigma = 0.02$). We utilize a cosine learning rate scheduler with a 2,000-step warmup for the initial training phase. The peak learning rates are set to 3×10^{-3} for the 170M models and 1.25×10^{-3} for the 760M models. Training is performed on the FineWeb-Edu dataset using the Flame framework (Zhang & Yang, 2025).

We distinguish the initialisation between standard training ($b_{scale} = 1$) and a fine-tuning regime ($b_{scale} \in [0.1, 1.0]$). In the fine-tuning stage, models resume from intermediate checkpoints with a shortened 200-step warmup. To ensure stability during this phase, the peak learning rate is reduced to 1/10 of the original value for both model scales. Models are trained for the final 1B (15B total budget) or 2B (30B total budget) tokens, maintaining a fixed computational budget across all experiments.

Table 5. Architectural and training details for language modeling experiments. Palimpsest-D and Palimpsest-M utilize consistent initialization schemes. Fine-tuning runs start from the corresponding pre-trained checkpoint.

Size	Model	Layers	Dim	Heads	E_k	E_v	Peak LR
170M	Transformer++	20	768	16	–	–	3.0e-3
	Gated Deltanet	19	768	16	1.0	2.0	
	Palimpsest-D	19	768	16	1.0	2.0	
	Palimpsest-M	30	768	16	1.0	2.0	
760M	Transformer++	25	1536	16	–	–	1.25e-3
	Gated Deltanet	19	1536	16	1.0	1.0	
	Palimpsest-D	19	1536	16	1.0	2.0	
	Palimpsest-M	38	1536	16	1.0	2.0	

A.10. Fine-Tuning Procedure for the 2.7B Model

For our large-scale experiments, we utilize the official [state-spaces/mamba2-2.7b](#) checkpoint as our base model. This architecture consists of 64 layers with a hidden dimension (d_{model}) of 2560 and a vocabulary size of 50,277. All undefined hyperparameters fall back to the default Mamba2 configuration.

As introduced in the main text, we analyze the layer-wise effective memory window N_t of the pre-trained Mamba2 model to determine the optimal placement for metaplasticity (Figure 6). We surgically replace the top 8 layers exhibiting the largest median N_t across the sequence dimension (i.e., the weakest forgetting) with trainable Palimpsest blocks, preserving the Mamba2 architecture for the remaining 56 layers.

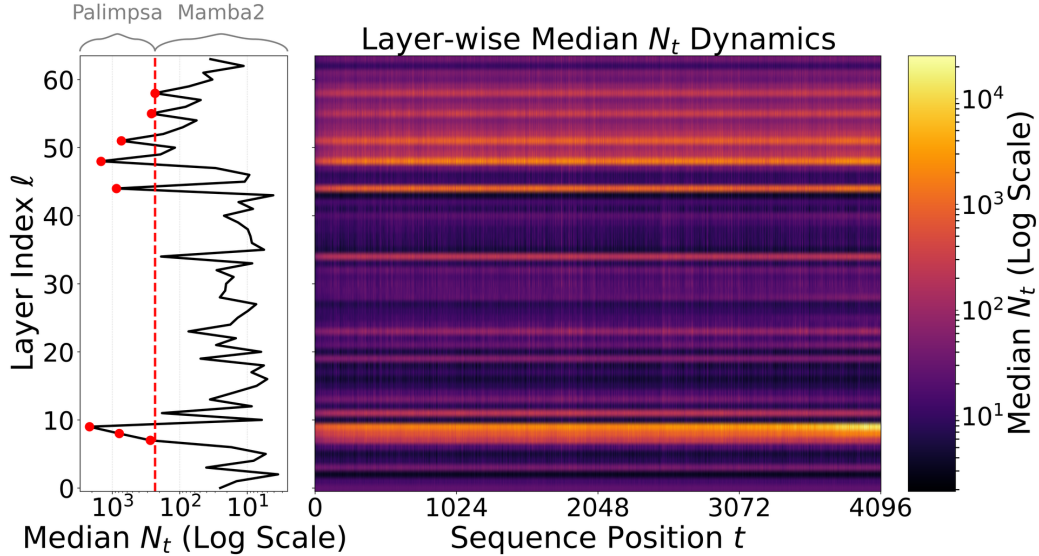


Figure 6. **Layer-wise Memory Window (N_t):** Median N_t computed across heads for each layer as a function of token position. The 8 layers exhibiting the highest sustained N_t (weakest forgetting, marked in red on the left panel) were selected for surgical replacement with Palimpsest layers.

After this substitution, we employ a three-stage training pipeline to adapt the two models for extended context lengths:

- **Stage 1: Continued Pre-training.** We trained the hybrid model for an additional 10B tokens on a mixture of FineWebEdu and StackV2.
- **Stage 2: Context Extension.** We extended the context window up to 32k tokens by training for 5B tokens on long-form text (Gao et al., 2025; Agarwalla et al., 2024). This dataset comprised the “arxiv” and “book” subsets of SlimPajamas, repository-level concatenations of StackV2 (C++ and Python code), and synthetic Needle-in-a-Haystack data to adapt the state to the extended time horizon.

- **Stage 3: Instruction Fine-Tuning.** Finally, the model was instruction fine-tuned on the LongAlpaca dataset (Chen et al., 2024) to align its outputs for downstream benchmark evaluations.

A.11. Ruler Benchmark Experiments

We present the full RULER (Hsieh et al., 2024) evaluation for our 2.7B parameter models in Table 6. RULER assesses distinct aspects of long-context performances.

In the Retrieval category, a clear divergence emerges. The non-metaplastic baseline dominates single-needle retrieval (N-1, N-2, N-3), perhaps because input selectivity can effectively isolate and protects a single vital piece of information. However, the fine-tuned Palimpsest variant consistently wins when retrieving multiple distinct keys (N-MQ), multiple values sharing a key (N-MV), or ignoring hard distractors (N-MK). This suggests metaplasticity provides a superior compromise between learning and remembering when multiple contextual pieces are important.

Crucially, the Variable Tracing (VT) results perfectly illustrate our core hypothesis. While the metaplastic 2.7B variant underperforms the baseline at shorter contexts (4k–16k), it exhibits a significantly lower degradation rate as sequence length increases. By the 32k context length, Palimpsest decisively surpasses the baseline (24.4 vs. 14.9), confirming its superior capacity to retain sequential dependencies in the extended regime.

For Aggregation tasks (CWE, FWE), the baseline retains an advantage. Metaplasticity appears to negatively impact the extraction of frequent words, though the exact mechanistic reason remains an open question. Finally, results on Question Answering (HotPot, SQA) remain mixed depending on the specific context length.

Table 6. Comprehensive RULER Benchmark evaluation of the surgically upgraded 2.7B parameter Palimpsest-M models across context lengths from 4k to 32k. Tasks are grouped by category to isolate specific memory behaviors. Best results are in bold.

Model	Retrieval						Tracing VT	Aggregation		QA	
	N-1	N-2	N-3	N-MQ	N-MV	N-MK		CWE	FWE	HotPot	SQA
4k Context											
Palimpse-M (w/o Meta)	100.0	88.0	83.6	53.1	51.5	25.0	59.6	30.5	64.0	25.8	28.8
Palimpse-M (Fine-tuned 15BT)	100.0	84.4	77.2	57.5	59.9	28.4	33.5	29.6	58.4	29.8	28.4
8k Context											
Palimpse-M (w/o Meta)	100.0	46.2	42.4	38.8	34.9	24.6	49.7	20.5	64.0	24.4	19.0
Palimpse-M (Fine-tuned 15BT)	100.0	44.8	42.4	39.4	37.2	21.8	24.9	13.0	56.7	23.8	19.2
16k Context											
Palimpse-M (w/o Meta)	100.0	9.4	13.6	7.0	6.0	8.0	26.0	2.4	59.6	18.4	15.6
Palimpse-M (Fine-tuned 15BT)	99.4	11.2	12.6	8.2	9.7	8.0	22.5	0.6	43.8	17.0	17.1
32k Context											
Palimpse-M (w/o Meta)	94.4	7.0	9.4	2.3	1.1	7.2	14.9	0.4	31.9	20.2	14.7
Palimpse-M (Fine-tuned 15BT)	93.0	7.6	8.2	2.8	1.4	7.6	24.4	0.2	24.7	20.6	15.1

A.12. Palimpsest Implementation and Parallelization

Our parallel training algorithm is implemented as a fused Triton kernel utilizing a chunk-wise associative scan. The core logic is structured as follows:

- **Chunk-wise Processing:** The input sequence is partitioned into chunks of size B_T .
- **Intra-Chunk Parallel Scan:** Within each chunk, the recursive state updates for the first moment M_t and importance I_t are computed in parallel using an associative scan.
- **Inter-Chunk Sequential Update:** The final state of one chunk is carried over as the initial state for the subsequent chunk to ensure global temporal consistency.

This approach maximizes GPU utilization by transforming the sequential recurrence into a parallelizable prefix-sum operation. The kernel also performs state checkpointing to maintain numerical stability during the backward pass.

To implement the parallel scan, we define the state as a tuple $(\mathbf{M}_t, \bar{\mathbf{I}}_t, A_t)$, where $\bar{\mathbf{I}}_t = \mathbf{I}_t - \mathbf{I}_{\text{prior}}$ represents the centered importance. Let $t \in [1, B_T]$ be the index within a chunk, and let $\mathbf{M}_0$ and $\mathbf{I}_0$ be the final states from the previous chunk. The states are computed via the associative operator $\oplus$:

$$(\mathbf{M}_b, \bar{\mathbf{I}}_b, A_b) \oplus (\mathbf{M}_a, \bar{\mathbf{I}}_a, A_a) = (\mathbf{M}_b + A_b \mathbf{M}_a, \bar{\mathbf{I}}_b + A_b \bar{\mathbf{I}}_a, A_a A_b) \quad (35)$$

The posterior mean is recovered as $\boldsymbol{\mu}_t = \mathbf{M}_t / \mathbf{I}_t$, followed by an *einsum* with the query $\mathbf{q}_t$ to produce the output.

A primary limitation of Palimpsest is the necessity to explicitly store all Intra-Chunk states $\mathbf{M}_t$ and $\mathbf{I}_t$ in registers to perform the output computation. This requirement imposes a heavy register memory footprint, creating a significant bottleneck. To mitigate this register pressure, we are constrained to use smaller chunk dimensions B_T , which negatively impacts training throughput.

Furthermore, all Inter-Chunk states must be materialized in HBM, which can restrict the maximum batch size during training as the state size and sequence length scale. This creates a fundamental trade-off: a larger B_T reduces HBM traffic but significantly increases register pressure.

In practice, the performance overhead is size-dependent: for a 170M parameter model, the training time is approximately $1.5\times$ that of an optimized Simple GLA from the FLA (Yang et al., 2024) kernel, while for a 760M model, this ratio increases to approximately $3\times$.

A.13. Inference Efficiency

While Palimpsest’s training throughput is constrained by register pressure from the parallel associative scan, these bottlenecks vanish at inference time. During generation, the model operates in its native recurrent mode, processing tokens sequentially. This eliminates the need to materialize intermediate states across large chunks, reducing the memory footprint to a constant state size per step.

We benchmarked the fused recurrent kernel on a single **NVIDIA GeForce RTX 3090** GPU ($B = 16, L = 2048, H = 1$), varying the model dimension $D \in \{512, 1024, 2048\}$. As shown in Figure 7, Palimpsest demonstrates robust throughput, peaking at roughly 770 kt/s for $D = 512$ and maintaining a consistent factor of $\approx 4\times$ slowdown compared to Simple GLA as the dimension scales to 2048 (~ 50 kt/s vs ~ 188 kt/s).

It is important to note that this benchmark isolates the attention mechanism. In the context of a full model inference pass, the relative performance gap would narrow significantly, as a substantial portion of the compute budget is consumed by the Gated MLP, embeddings, and layer normalization, which are identical across both architectures.

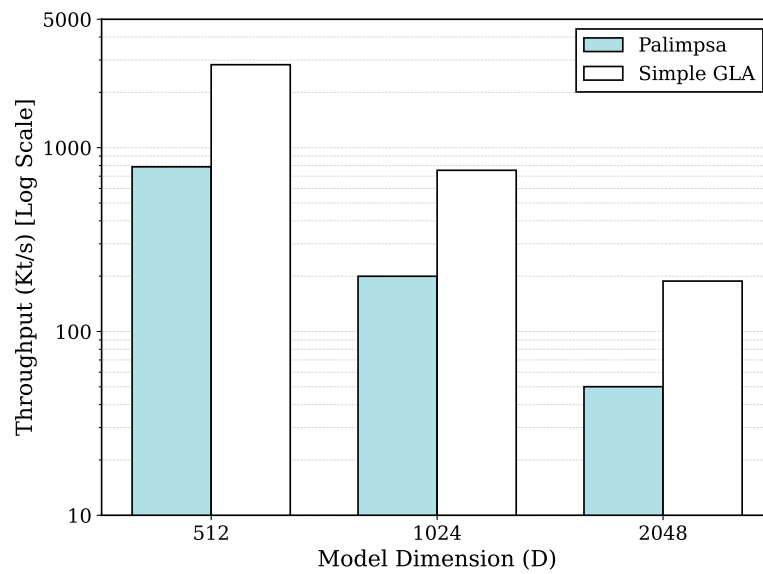


Figure 7. Inference Speed Benchmark. Throughput (thousands of tokens/s) comparison between Palimpsa and Simple GLA on an NVIDIA GeForce RTX 3090. Palimpsa matches the baseline’s scaling behavior while maintaining a consistent $4\times$ factor due to the dual-state update overhead.